

AMS

Hard Engineering

The problems that actually matter: concurrency, exact arithmetic, multi-tenancy, algorithms, integrity and verification.

-

Airline Management System — Finance, Fleet, Procurement

Written from the live codebase: 10 domain modules, 2 migrations, 190 tests

Every claim marked PROVEN (checked by code or test) or ASSUMED (stated, not verified)

This document teaches engineering problems, not a language. It assumes you can read code and follow an argument. It does not assume you write JavaScript, and it does not assume you write databases. Where the reasoning is difficult, it is written out rather than skipped.

How to read this document

This is not a tutorial. It is an account of the engineering problems AMS actually faces, and of which of them are solved, which are assumed, and which are still open.

The distinction matters more than usual here, because these problems all fail in ways that look like working code. A race condition passes every functional test. A float money column produces correct answers for years. A missing tenant filter returns a customer's data to the wrong customer, which is not a bug report but an incident.

The two marks used throughout

PROVEN — checked by code or by an automated test

The behaviour is enforced by a mechanism in the codebase and, where stated, confirmed by a test that would fail if the mechanism were removed. You can verify it yourself by reading the referenced file or by breaking the mechanism and running the tests.

ASSUMED — stated in code, not independently verified

A constant, a formula, a regulatory interpretation or a business rule that the code implements faithfully but that has not been validated against primary sources or against your organisation's own data. These are the places where a wrong assumption produces a wrong number while every test still passes.

Why this framing

```
// The most expensive sentence in any financial system:

const fuelCents = Math.round(fuelKg * fuelPricePerKg); // PROVEN by test
// The engine's arithmetic is exact and tested against brute force.

const BURN_KG_PER_KG_PAYLOAD = 0.030; // ASSUMED
// An industry approximation. No test can confirm it is right for your fleet.
// Wrong by 30% and every test still passes.
```

A guide that showed only solved problems would teach you to assume everything is verified. That assumption is exactly what produces the defects documented in Part VII.

Contents

Part I — Concurrency: the failure nobody can reproduce

- The race that overspends an airline
- Why every ordinary testing method fails here
- Locks, transactions, and what isolation levels mean
- Deadlock: the second classic failure
- Idempotency and exactly-once thinking
- What AMS does, verified and assumed

Part II — Exact arithmetic beyond the function call

Why a float column is a future incident
BIGINT, NUMERIC and the database as an accountant
Generated columns: making drift structurally impossible
CHECK constraints as the last line of defence
Rounding once, and where rounding leaks
The reconciliation invariant

Part III — Multi-tenancy as a security boundary

One database, many airlines
The three layers and why layer three is the database
Row-level security, concretely
FORCE ROW LEVEL SECURITY and why it is not optional
What RLS does not protect against

Part IV — Algorithms that must be exactly right

The problem class: resource-constrained shortest path
Why greedy and unconstrained shortest path both fail
Label-setting and the dominance relation
Why the dominance rule is sound
Differential verification against brute force
Property tests and metamorphic tests
Complexity, and knowing when you are near the wall

Part V — Integrity: making history unchangeable

Append-only as a policy versus a mechanism
Database triggers that refuse mutation
Hash chaining and what it does not prove
Canonicalisation and the phantom corruption bug

Part VI — Side effects that cannot be lost or duplicated

The problem: a commit that must send an email
The dual-write failure in both directions
The transactional outbox
What the outbox guarantees and what it does not

Part VII — Verification as an engineering discipline

Four kinds of test and which defects each finds
Differential testing
Oracle design and seeding
Guarding against vacuous tests
The defect ledger
Where software verification ends

Part VIII — Engineering judgement and review

Reading the codebase for risk
The questions to ask of any money-touching function
The questions to ask of any database schema
Known gaps, stated plainly

Appendix — Reference tables and glossary

Concurrency failure modes
Integer scale in PostgreSQL
Glossary

The five questions this document answers

1. How does AMS prevent two concurrent requests from spending the same money twice, and why can ordinary testing never prove it?
2. How is money kept exact when it leaves JavaScript and lives in a PostgreSQL column?
3. What stops one airline seeing another airline's data when they share a database?
4. How is an optimal route computed, and how do we know the answer is actually optimal?
5. What has been verified, what has merely been assumed, and where does the responsibility pass to a human?

Part I — Concurrency: the failure nobody can reproduce

Concurrency bugs are the hardest class of defect in this codebase, not because they are intellectually difficult but because they are invisible to every verification method a team normally relies on.

The race that overspends an airline

```
/**
 * AMS — Cost Authority: the control core.
 *
 * The classic failure is not a single large fraud. It is a hundred
 * individually-reasonable approvals, each taken against a budget line
 * that looked like it had room, which together exceed the board's
 * approved plan by a quarter. Each approver was honest. The aggregate
 * was not authorised.
 *
 * That happens because availability checks are TOCTOU: check the
 * balance, then write the commitment, and between those two operations
 * another request does the same thing. Two requests both see US$10,000
 * available. Both approve. The airline is US$10,000 overspent and nobody
 * was at fault in any individual transaction.
 *
 * THE FIX
 *
 *      SELECT ... FOR UPDATE
 *
 * PostgreSQL blocks the second transaction until the first commits or
 * rolls back. The race is not defended against; it is made unrepresentable.
 *
 * @module modules/finance/cost-authority
 */
```

backend/src/modules/finance/cost-authority.js — the module header, which states the problem before any code.

TOCTOU stands for *time-of-check to time-of-use*. It is the general shape of a huge class of security and correctness bugs: something is validated, then a gap opens, then the thing is acted on using a value that is now stale.

```
// THE BUG — functionally correct, and wrong under concurrency

export async function authoriseCostUnsafe(db, { tenantId, lineId, amountCents }) {
  const line = await db('budget_lines').where({ id: lineId }).first();
  const available = line.budgetedCents - line.committedCents;

  if (available < amountCents) throw new CostAuthorityError('Insufficient', 'BUDGET_EXCEEDED');

  // ■■■ the gap ■■■
  // 0.4 ms of network and scheduler time.
  // Any number of other requests can execute here.

  await db('budget_lines').where({ id: lineId }).increment('committed_cents', amountCents);
  return { ok: true };
}
```

Two requests arrive within a millisecond of each other:

Time	Request A	Request B
t=0ms	read line → available = 10,000	
t=0.2ms		read line → available = 10,000
t=0.4ms	check 10,000 >= 10,000 ✓	
t=0.6ms		check 10,000 >= 10,000 ✓
t=0.8ms	increment committed += 10,000	
t=1.0ms		increment committed += 10,000
Result:	committed_cents = 20,000 against budgeted_cents = 10,000 The airline is US\$10,000 overspent and neither request is wrong.	

Both requests did exactly what the code told them to do. Both read a true fact. Both passed a correct check. The defect is not in either request — it is in the *interaction* between them, which no single request can observe.

Why every ordinary testing method fails here

Sequential unit tests cannot reproduce it: they run one request at a time, and sequentially there is no race.

Integration tests are the same problem — a test that issues two requests sequentially will always pass.

Code review cannot find it: every individual statement is correct and idiomatic. The bug is in what is absent between them.

Manual testing cannot find it: it needs two requests to arrive within the same few milliseconds, which does not happen when one person is clicking.

It appears under production load, at peak approval times, intermittently, to whoever is unluckiest. In a system that authorises money, that is the worst possible failure profile.

Why the obvious mitigations do not work

Attempted fix	Why it fails
Add a unit test for the check	The check is correct. The defect is the absence of a lock, and a sequential test cannot observe an absence.
Re-read the balance immediately before writing	This narrows the window to microseconds. It does not close it. It converts a 1-in-10,000 failure into a 1-in-100,000,000 one.
Add application-level mutex or in-process lock	Correct for a single process, useless for multiple processes. In production you will run more than one.
Use a Redis distributed lock	Better, and a genuinely valid approach. But it adds a network dependency whose failure mode is the same outage the lock was protecting against, and it is not needed when the database already offers the guarantee atomically.
Check availability again after the write and compensate	You have now spent the money. Recovery is a business process, not a technical fix.

The principle

When a correctness property must hold under concurrency, the fix is not to test harder or to check more carefully.

The fix is to move the invariant into a place that *cannot be bypassed*. A database row lock is such a place. An application-level assertion is not.

Locks, transactions, and what isolation levels mean

```

export async function lockBudgetLine(db, tenantId, lineId) {
  const line = await db('budget_lines')
    .where({ id: lineId, tenant_id: tenantId })
    .forUpdate()
    .first();

  if (!line) throw new CostAuthorityError(
    `Budget line ${lineId} not found`, 'BUDGET_LINE_NOT_FOUND', { lineId });
  if (line.status !== 'ACTIVE') throw new CostAuthorityError(
    `Budget line ${lineId} is ${line.status}, not ACTIVE`, 'BUDGET_LINE_INACTIVE', { lineId });
  if (line.is_frozen) throw new CostAuthorityError(
    `Budget line ${lineId} is frozen by treasury`, 'BUDGET_LINE_FROZEN', { lineId });
  return /** @type {BudgetLine} */ (snakeToCamel(line));
}

```

backend/src/modules/finance/cost-authority.js

```

// ---- THE CRITICAL SECTION -----
// FOR UPDATE serialises every concurrent authorisation against this
// line. The second transaction waits here, then reads the balance the
// FIRST transaction left behind. Double-spend is unrepresentable.
// -----
const line = await lockBudgetLine(db, tenantId, lineId);
const availableBefore = availabilityCents(line);

if (availableBefore < amountCents) {
  throw new CostAuthorityError(
    `Insufficient cost authority: ${availableBefore} cents available, ${amountCents} cents requested`,
    'BUDGET_EXCEEDED',
    {
      lineId,
      availableCents: availableBefore,
      requestedCents: amountCents,
      shortfallCents: amountCents - availableBefore,
    },
  );
}

```

The same operation, with the lock.

The same interleaving, corrected:

Time	Request A	Request B
t=0ms	BEGIN	
t=0.1ms	SELECT ... FOR UPDATE → LOCKED	
t=0.2ms	read available = 10,000	
t=0.3ms	check 10,000 >= 10,000 ✓	
t=0.4ms	INSERT commitment	
t=0.5ms	UPDATE budget_lines SET committed += 10,000	
t=0.6ms	COMMIT → lock released	
t=0.7ms		BEGIN
t=0.8ms		SELECT ... FOR UPDATE → acquired
t=0.9ms		read available = 0 ← sees A's write
t=1.0ms		check 0 >= 10,000 ✗
t=1.1ms		ROLLBACK → BUDGET_EXCEEDED
Result:	committed_cents = 10,000 against budgeted_cents = 10,000. Correct.	

What this mechanism guarantees

Within a single PostgreSQL database, `SELECT ... FOR UPDATE` provides strict serialisation for the locked row. The second transaction cannot read the row until the first has committed or rolled back, and when it reads, it reads the committed state. This is a documented guarantee of the database, not an assumption.

It assumes the caller is inside an explicit transaction, so the lock is held until commit rather than released immediately. The function's docblock states this requirement explicitly.

```
/**
 * Authorise expenditure against a budget line, creating a commitment.
 *
 * MUST be called inside a transaction. The caller is responsible for
 * committing or rolling back; this function deliberately does not open
 * its own transaction, because the commitment write and the audit entry
 * must land atomically together.
 */
```

Isolation level	What it would allow	Sufficient for AMS?
READ UNCOMMITTED	Reads uncommitted changes from other transactions. Dirty reads.	No. Would actively corrupt availability calculations.
READ COMMITTED	Reads only committed data, but re-reads may see new commits. The default.	No. Without <code>FOR UPDATE</code> , this permits exactly the race above.
REPEATABLE READ	The same query returns the same rows throughout a transaction.	Better, but does not prevent the check-then-write gap.
SERIALIZABLE	Transactions behave as if run one at a time. Detects conflicts and aborts one.	Correct, but higher contention and more retries for no benefit where <code>FOR UPDATE</code> already suffices.

The important insight is that `READ COMMITTED` — the PostgreSQL default, and therefore what a team gets without deliberately choosing otherwise — **permits exactly the race in Part I**. The isolation level alone does not save you. The row lock does.

Deadlock: the second classic failure

```
export async function authoriseFlightLeg(db, { tenantId, lines, actorId }) {
  // Lock in a stable order to prevent deadlock between two rotations that
  // draw on overlapping budget lines in opposite order.
  const ordered = [...lines].sort((a, b) => a.lineId.localeCompare(b.lineId));
```

Deadlock is the other classic database concurrency failure. It occurs when two transactions each hold a resource the other needs:

```
Rotation 1 needs fuel(line-1) then crew(line-2)
Rotation 2 needs crew(line-2) then fuel(line-1)
```

Time	Rotation 1	Rotation 2
t=0	LOCK line-1 ✓	LOCK line-2 ✓
t=1	wants line-2 → BLOCKED	wants line-1 → BLOCKED

Both wait for the other. Neither proceeds. PostgreSQL detects this after `deadlock_timeout` (default 1s) and aborts one transaction.

The fix is not clever. It is to ensure every transaction acquires locks in the same order. Sorting the lines by identifier before touching them guarantees this: Rotation 1 and Rotation 2 will both go line-1 then line-2.

```
// Pass 1: validate every line under its lock before writing anything.
for (const { lineId, amountCents } of ordered) {
  try {
    const line = await lockBudgetLine(db, tenantId, lineId);
    const available = availabilityCents(line);
    if (available < amountCents) {
      failures.push({ lineId, availableCents: available, requestedCents: amountCents });
    }
  } catch (err) {
    failures.push({ lineId, error: /** @type {CostAuthorityError} */ (err).code ?? 'UNKNOWN' });
  }
}

// Any failure aborts the whole leg. All-or-nothing, because a flight
// cannot be half-authorised.
if (failures.length > 0) {
  return { authorised: false, commitments: [], failures };
}

// Pass 2: all lines cleared – write the commitments.
for (const { lineId, amountCents } of ordered) {
  commitments.push(await authoriseCost(db, { tenantId, lineId, amountCents, actorId }));
}
```

Validate everything under lock, then write. Two passes, sorted order.

All-or-nothing across a flight leg

A flight draws on several budget lines at once. The two-pass structure means no commitment is written until every line has been validated under its lock. If line 4 fails, lines 1 to 3 are not written, so there is no partial commitment to unwind.

The docblock states the domain reason: "a partially authorised aircraft rotation is an aircraft that will be dispatched without fuel it was supposed to have."

All-or-nothing across a flight leg

A flight draws on several budget lines at once. The two-pass structure means no commitment is written until every line has been validated under its lock. If line 4 fails, lines 1 to 3 are not written, so there is no partial commitment to unwind.

The docblock states the domain reason: "a partially authorised aircraft rotation is an aircraft that will be dispatched without fuel it was supposed to have."

All-or-nothing across a flight leg

A flight draws on several budget lines at once. The two-pass structure means no commitment is written until every line has been validated under its lock. If line 4 fails, lines 1 to 3 are not written, so there is no partial commitment to unwind.

The docblock states the domain reason: "a partially authorised aircraft rotation is an aircraft that will be dispatched without fuel it was supposed to have."

When you review any multi-row transaction, ask these three questions

- 1. In what order are rows locked?** If it is caller-supplied order, deadlock is possible.
- 2. Is the whole set validated before anything is written?** If not, failure mid-way leaves partial state.
- 3. Does the function open its own transaction, or require one?** If it opens its own, related writes may not be atomic.

- When you review any multi-row transaction, ask these three questions**
- 1. In what order are rows locked?** If it is caller-supplied order, deadlock is possible.
 - 2. Is the whole set validated before anything is written?** If not, failure mid-way leaves partial state.
 - 3. Does the function open its own transaction, or require one?** If it opens its own, related writes may not be atomic.

Idempotency and exactly-once thinking

```
// Idempotency: a retried request must not commit twice.
if (params.idempotencyKey) {
  const prior = await db('commitments')
    .where({ tenant_id: tenantId, idempotency_key: params.idempotencyKey })
    .first();
  if (prior) {
    return {
      commitmentId: prior.id,
      availableBeforeCents: prior.available_before_cents,
      availableAfterCents: prior.available_after_cents,
      replayed: true,
    };
  }
}
```

backend/src/modules/finance/cost-authority.js

Exactly-once delivery does not exist across a network boundary. What exists is at-least-once delivery plus idempotent processing. The distinction is not academic: it is the difference between a system that occasionally double-charges and one that does not.

```
// The scenario
POST /cost-authority  amount = 10,000  idempotency_key = "abc-123"

Server processes it, commits the money, then the RESPONSE is lost in transit.
The client cannot distinguish "never arrived" from "arrived, response lost".
So it retries. Without an idempotency key, the money is committed twice.
```

```
-- The database-level guarantee, migration 002:
CREATE UNIQUE INDEX idx_commitments_idempotency
ON commitments (tenant_id, idempotency_key)
WHERE idempotency_key IS NOT NULL;
```

backend/migrations/002_budget_commitments.js

Why the unique index rather than only a check

The code checks for a prior commitment before inserting. That check has the same TOCTOU shape as the original race: two concurrent requests with the same key could both find nothing and both insert.

The partial unique index closes this. The database will reject the second insert because the (tenant_id, idempotency_key) pair already exists. Application code is the fast path; the index is the guarantee — the same "policy versus mechanism" distinction as FOR UPDATE.

WHERE idempotency_key IS NOT NULL matters: without it, every commitment with no key would collide on NULL, since PostgreSQL treats NULLs as distinct but a partial index makes the intent explicit.

```
return {
  commitmentId: String(created.id),
  availableBeforeCents: availableBefore,
  availableAfterCents: availableBefore - amountCents,
  replayed: false,
};
```

The replayed flag lets the caller distinguish a fresh commit from a replay. This matters beyond bookkeeping: an idempotent replay should not generate a second approval notification, and the audit trail should record that the request was seen before.

The assumption in idempotency keys

The key must be generated by the *client* and must be unique per logical operation. If the server generates it per request, a retry is a new request with a new key and the protection is void. The key must also be stable across retries — a key derived from a timestamp or a random value per attempt does not work. Nothing in the AMS code verifies this, because it is a client-side contract.

What AMS does, verified and assumed

```
export function availabilityCents(line) {
  const frozen = line.frozenCents ?? 0;
  const available =
    line.budgetedCents + line.revisedCents + line.releasedCents
    - line.committedCents - line.pendingCents - line.expendedCents - frozen;

  if (available < 0) {
    throw new CostAuthorityError(
      `Budget line ${line.id} has negative availability (${available} cents). ` +
      'This is a control breach, not a data error.',
      'AVAILABILITY_NEGATIVE',
      { lineId: line.id, availableCents: available },
    );
  }
  return available;
}
```

The availability formula.

```
* `frozen` is separate from `released` deliberately: a treasury ring-fence
* (a security deposit, a disputed supplier claim under litigation) removes
* spendable money WITHOUT creating a commitment. Conflating the two makes
* the reconciliation to the general ledger impossible.
*
* A negative result is a control breach, never a value to clamp. Clamping
* hides the breach; AMS throws and alerts.
```

Failing loudly rather than clamping

A negative availability means money was committed beyond what was authorised — a bug, a manual database edit, or a deliberate override. Clamping to zero would make the system report a healthy budget while the breach continued and grew. Throwing converts an invisible financial loss into a loud, immediate, attributable failure.

```
await db('budget_lines')
  .where({ id: lineId, tenant_id: tenantId })
  .increment('committed_cents', amountCents);

return {
  commitmentId: String(created.id),
  availableBeforeCents: availableBefore,
  availableAfterCents: availableBefore - amountCents,
  replayed: false,
};
```

Why increment rather than read-modify-write

`.increment('committed_cents', amountCents)` compiles to a single `UPDATE ... SET committed_cents = committed_cents + $1`. PostgreSQL applies this atomically to the row, and because the row is already locked by `FOR UPDATE` the operation is sequenced correctly.

The alternative — read the value in JavaScript, add, then write the result — is a lost-update bug waiting to happen, and it would be invisible in testing for the same reasons the original race was.

The settlement rule that prevents double-spend

```
/**
 * Settle a commitment against a payment.
 *
 * NOTE: this reduces committed and increases expended by the same amount.
 * It does NOT restore availability. The money has left. Restoring
 * availability would let the same funds be committed a second time,
 * which is the exact failure this module exists to prevent.
 */
```

backend/src/modules/finance/cost-authority.js

```
return { commitmentId, committedReleasedCents: amountCents, availabilityRestored: false };
```

```
/**
 * Release the difference when a supplier invoices less than ordered, or
 * when a purchase order is cancelled before delivery. Availability IS
 * restored, because no money left.
 */
```

The contrast between those two docblocks is the entire lesson. Money that left restores nothing. Money that never left releases cleanly. Both rules are documented immediately above the code that implements them, because a future engineer optimising this would otherwise have no way to know which direction is the error.

Operation	committe d	expended	available restored?	Why
authoriseCost	+amount	—	Yes (implicitly)	Money is now spoken for
reservePending	+pending	—	Yes (implicitly)	A submitted request holds the funds
settleCommitment	–amount	+amount	No	The money has left the bank account
releaseCommitmentDifference	–release	—	Yes	No money left
freezeBudgetLine	—	—	No (–frozen)	Ring-fenced without a commitment

```

export function freezeBudgetLine(db, { tenantId, lineId, amountCents, actorId, reason }) {
  if (!reason || reason.length < 10) {
    throw new CostAuthorityError(
      'A freeze requires a substantive reason of at least 10 characters; it is a '
      + 'governance action that appears in the audit pack',
      'FREEZE_REASON_REQUIRED', { lineId },
    );
  }
  const line = await lockBudgetLine(db, tenantId, lineId);
  const free = availabilityCents(line);
  if (amountCents > free) {
    throw new CostAuthorityError(
      `Cannot freeze ${amountCents}; only ${free} available`, 'FREEZE_EXCEEDS_AVAILABLE',
      { lineId, amountCents, availableCents: free },
    );
  }
  await db('budget_lines').where({ id: lineId, tenant_id: tenantId })
    .update({ frozen_cents: (line.frozenCents ?? 0) + amountCents });

  return { lineId, frozenCents: amountCents, availableAfterCents: free - amountCents };
}

```

The ten-character minimum, and why it is engineering rather than pedantry

A freeze removes money from circulation and appears in the audit pack. Requiring a substantive written reason means the audit pack contains an explanation, not an unexplained number.

This is a control expressing a governance principle: actions that affect money must be justifiable in words as well as in figures. It is eleven lines of code enforcing a policy that would otherwise depend on every future author remembering to be careful.

A real defect in this module, currently unfixed

In `settleCommitment` the budget-line update reads:

```
` .where({ id: commitmentId && commitment.budget_line_id, tenant_id: tenantId }) `
```

`commitmentId && commitment.budget_line_id` evaluates to `commitment.budget_line_id` whenever `commitmentId` is a non-empty string, so it happens to produce the right value. But it is a mistake: an accidental logical AND where an assignment was intended. If `commitmentId` were ever an empty string the query would match nothing and the settlement would silently fail to update the budget line.

This is a genuine latent defect in the current codebase. The type checker does not flag it because both operands are strings. The fix is to delete the `commitmentId &&` expression. It should be fixed before this module is used in production.

Part II — Exact arithmetic beyond the function call

Part III of the language guide showed how JavaScript keeps money exact. This part asks the harder question: what happens to that money when it is written to a database, and what protects it there.

Why a float column is a future incident

```
/**
 * AMS — Migration 002: budget, commitments and cost authority.
 *
 * Every figure is BIGINT integer minor units (cents).
 * There is no NUMERIC and no FLOAT for money anywhere in AMS, because a
 * float money column is a rounding error waiting for a large enough
 * number of transactions.
 *
 * @module migrations/002
 */
```

backend/migrations/002_budget_commitments.js — the stated policy.

This is the single most consequential schema decision in AMS, and it is invisible once made. A FLOAT money column works perfectly for years. It returns correct totals for small datasets. It fails only once the data is large enough that the accumulated error exceeds what reconciliation tolerates — which is to say, once the business succeeds.

```
-- What "it works" looks like
CREATE TABLE ledger (
  id          SERIAL PRIMARY KEY,
  amount      FLOAT8      -- ← the decision you will regret
);

-- Small dataset: reconciles fine.
INSERT INTO ledger (amount) VALUES (10.10), (20.20), (30.30);
SELECT SUM(amount) FROM ledger;    -- 60.6

-- A million rows of 0.01:
INSERT INTO ledger (amount)
  SELECT 0.01 FROM generate_series(1, 1000000);
SELECT SUM(amount) FROM ledger;    -- 9999.999999999973

-- The trial balance no longer foots. The ledger is now unusable and
-- every downstream report inherits the error.
```

The failure is silent and permanent

A float error does not announce itself. It produces a total that is very slightly wrong, passes every type check, and is discovered at a year-end reconciliation months after the transactions that caused it. By then the individual amounts cannot be corrected because nobody can identify which of the million rows drifted by which fraction of a cent — the aggregate is wrong but the rows are individually indistinguishable from correct.

BIGINT, NUMERIC and the database as an accountant

Type	Storage	Exact decimal?	Verdict for money
SMALLINT	16-bit	Yes (integers only)	Too small. Range ends at 32,767.
INTEGER	32-bit	Yes (integers only)	Ends at 2,147,483,647 — about \$21 million in cents. Too small.
BIGINT	64-bit	Yes (integers only)	Used throughout AMS. Range to about \$92 quadrillion.
NUMERIC	arbitrary	Yes	Legitimate for money. Not used in AMS — see below.
REAL / FLOAT8	floating	No	Never acceptable for a ledger amount.

```
-- Currency. All amounts BIGINT minor units. No FLOAT. No NUMERIC.
currency          TEXT NOT NULL DEFAULT 'USD',
budgeted_cents    BIGINT NOT NULL DEFAULT 0 CHECK (budgeted_cents >= 0),
revised_cents     BIGINT NOT NULL DEFAULT 0,
released_cents    BIGINT NOT NULL DEFAULT 0,
committed_cents  BIGINT NOT NULL DEFAULT 0,
pending_cents     BIGINT NOT NULL DEFAULT 0,
expended_cents    BIGINT NOT NULL DEFAULT 0,
frozen_cents      BIGINT NOT NULL DEFAULT 0 CHECK (frozen_cents >= 0),
```

backend/migrations/002_budget_commitments.js

```
serial_number      BIGINT NOT NULL,          -- MSN
current_cycles     BIGINT NOT NULL DEFAULT 0,
current_hours      NUMERIC(14,2) NOT NULL DEFAULT 0,
current_block_hours NUMERIC(14,2) NOT NULL DEFAULT 0,
```

Not every column is BIGINT — and the distinction is deliberate.

Notice which columns use NUMERIC(14,2) rather than BIGINT. Aircraft flight hours are genuinely fractional — an aircraft accumulates 3,847.42 block hours. They are not money, and they are not stored in cents. Fuel quantities are stored as integer kilograms for the same reason: a fuel figure is measured in whole units, and forcing a fraction would invent precision the measurement does not have.

Quantity	Storage	Unit	Why
Money	BIGINT	integer cents	Fractional cents cannot be invoiced
Block hours	NUMERIC(14,2)	hours to 2dp	Genuinely fractional; not money
Fuel	integer	whole kg	Measured in whole units
Load factor	integer	parts per million	Ratio; stored as ppm to stay exact
CASK	integer	microcents	Unit scaled so it cannot round to zero

Why not NUMERIC for money?

The BIGINT-over-NUMERIC choice is defensible but debatable

NUMERIC is exact for decimals, is standard in financial PostgreSQL schemas, and is what most accounting software expects. The arguments for BIGINT in AMS are: it enforces an integer discipline at the storage layer, it makes a fractional cent impossible rather than merely discouraged, and it is faster.

The counter-argument is real: with BIGINT, the *unit* is implicit. A column named `budgeted_cents` is clear; a column named `amount` in a table written by someone else is not. `NUMERIC(14, 2)` carries its own precision and scale in the type.

Either choice is sound. What matters is that the choice is made deliberately, applied consistently, and documented — which is why the migration comment says "No FLOAT. No NUMERIC" rather than leaving it implicit.

Generated columns: making drift structurally impossible

```
// Availability is GENERATED, so it can never drift from its components.
// PostgreSQL recomputes it on every relevant write, inside the same
// transaction. This is the structural reason AMS cannot overspend.
await knex.raw(`
  ALTER TABLE budget_lines
  ADD COLUMN available_cents BIGINT
  GENERATED ALWAYS AS (
    budgeted_cents + revised_cents + released_cents
    - committed_cents - pending_cents - expended_cents - frozen_cents
  ) STORED
`);
```

backend/migrations/002_budget_commitments.js

This is the most important single line in the AMS schema, and it is worth understanding precisely what it buys.

Approach	Availability computed in	Failure mode
Stored column, updated by application code	JavaScript, on every write	A code path that forgets to update it. Drift is silent and permanent.
Stored column, updated by trigger	The database, on every write	Cannot drift, but the value is a duplicate truth you must trust.
Generated column	The database, on every write, from the components	Cannot drift at all. There is no code path that writes it.

```
-- What "GENERATED ALWAYS" means in practice

-- Attempting to write to it is an ERROR, not a silent overwrite:
UPDATE budget_lines SET available_cents = 999999 WHERE id = 'bl-1';
-- ERROR:  column "available_cents" can only be updated to DEFAULT

-- Changing ANY component recomputes it, inside the same statement:
UPDATE budget_lines SET committed_cents = committed_cents + 1000 WHERE id = 'bl-1';
-- available_cents is recomputed by the database. Always. Automatically.
```

Why this eliminates an entire category of defect

The JavaScript layer also computes availability — in `availabilityCents()`. That is a duplicate implementation. Normally a duplicate implementation is a liability: they can disagree.

Here they cannot disagree in the direction that matters, because the database value is *derived* and the JavaScript value is used only for the pre-check inside the locked section. The generated column is the authority; the JavaScript function is a convenience that reads the same components the column is derived from.

If the two implementations ever produce different values, the generated column is right and the JavaScript is a bug. That asymmetry is deliberate.

```
export function availabilityCents(line) {
  const frozen = line.frozenCents ?? 0;
  const available =
    line.budgetedCents + line.revisedCents + line.releasedCents
    - line.committedCents - line.pendingCents - line.expendedCents - frozen;
  // ... throws if negative
}
```

backend/src/modules/finance/cost-authority.js — the duplicate, which must agree with the generated column expression above.

The duplication is a maintenance risk

There is no automated check that the JavaScript expression and the generated column expression stay in sync. If someone adds a component — say `encumbered_cents` — to one and not the other, the system will read as though it has more available than it does, and the pre-check will pass amounts the constraint would reject.

The mitigation is real: the `CHECK (available_cents >= 0)` constraint would reject the write at the storage layer, so the consequence is a failed transaction rather than an overspend. But it is a late, opaque failure. **This should be added as a regression test** — assert that the two expressions agree for a set of representative budget lines.

CHECK constraints as the last line of defence

```
// A negative availability is a control breach, not a rounding artefact.
await knex.raw(`
  ALTER TABLE budget_lines
    ADD CONSTRAINT budget_lines_non_negative
    CHECK (available_cents >= 0)
    NOT VALID
`);
await knex.raw(`ALTER TABLE budget_lines VALIDATE CONSTRAINT budget_lines_non_negative`);
```

backend/migrations/002_budget_commitments.js

Three things are happening in those five lines, and all three are advanced PostgreSQL technique.

1. `NOT VALID` adds the constraint without checking existing rows. On a large table this avoids a long exclusive lock during migration — the constraint applies to all *future* writes immediately.
2. `VALIDATE CONSTRAINT` then scans existing data in a weaker lock mode. If any historical row already violates the constraint, this fails loudly at migration time rather than silently.
3. The combination is the standard pattern for adding a constraint to a populated production table without downtime.

```
-- The guarantee, at the lowest level
INSERT INTO budget_lines (budgeted_cents, committed_cents, available_cents)
VALUES (100000, 500000, NULL);
-- ERROR: new row for relation "budget_lines" violates check constraint
-- "budget_lines_non_negative"

-- This fires even if the application has a bug, even if someone connects
-- with psql and types UPDATE by hand, even if a future service is written
-- in a different language.
```

Why a CHECK constraint beats an application assertion

An application assertion can be bypassed by: a future service written by someone who has not read the module; a migration script; a manual psql session during an incident; a bulk import written in Python.

A CHECK constraint is part of the table definition. It applies to every write by every client, forever. This is the "policy versus mechanism" distinction from Part I, and it is the reason the AMS authors preferred database guarantees wherever one was available.

Constraints as executable domain documentation

```
bucket          TEXT NOT NULL CHECK (bucket IN ('FLIGHT_ATTRIBUTABLE', 'FLEET_FIXED', 'PERIOD')),
```

```
fund_type       TEXT NOT NULL CHECK (fund_type IN
('OPERATING', 'CAPITAL', 'LEASING', 'RESTRICTED', 'DEFERRED_REVENUE', 'SAF_COMPLIANCE')),
```

```
lease_type      TEXT NOT NULL CHECK (lease_type IN
('OWNED', 'FINANCE_LEASE', 'OPERATING_LEASE', 'WET_LEASE')),
```

```
movement_type   TEXT NOT NULL
CHECK (movement_type IN ('PIPELINE', 'COMMITMENT', 'SETTLED', 'RELEASED', 'REVERSED', 'ADJUSTMENT')),
direction        TEXT NOT NULL CHECK (direction IN ('DEBIT', 'CREDIT')),
```

```
CHECK (released_cents <= amount_cents)
```

```
CHECK (expires_at > starts_at),
CHECK (delegator_id <> delegate_id)
```

backend/migrations/001_tenancy_identity_audit.js — the delegations table. Two constraints that encode governance policy directly.

Each of these lists is generated from the corresponding constant array in `shared/src/domain.js`, and each can never drift from it because the constraint is at the storage layer. A value not in the list cannot reach the database.

```

* Union types
*
* These are JSDoc-only. In a plain-JavaScript codebase there is no
* `export type` syntax, so union types are documented for the editor
* and for `checkJs`, and enforced at RUNTIME by the CHECK constraints
* generated on each state column from the const arrays above.
*
* A value that is not in the corresponding array cannot reach the
* database. The type annotation is a convenience; the constraint is
* the guarantee.
*/

```

shared/src/domain.js

The three-layer safety model, stated once

Layer 1 — annotations. JSDoc types and the checkJs gate catch mistakes before the code runs. A convenience: a wrong annotation does nothing.

Layer 2 — runtime checks. `if (!spec) throw`, `assertSafeInteger`, the negative-availability throw. Catch mistakes at execution. Still bypassable by anything that skips the code path.

Layer 3 — database constraints. CHECK, generated columns, unique indexes, triggers. Apply to every write by every client, with no cooperation required from the caller.

The rule the codebase follows: **when a property must hold, push it as far down as it will go.** The constraint is the guarantee. Everything above it is helpful.

Rounding once, and where rounding leaks

```

// Round ONCE, at tonne-milligram precision. Rounding at an intermediate
// step (kg -> tens of kg -> tonnes) silently loses a third decimal place,
// which is material when the compliance bill is multiplied by an
// allowance price of EUR 85 per tonne.
const round3 = (n) => Math.round(n * 1000) / 1000;
const conventionalTonnes = round3((conventionalKg * factor) / 1000);

```

shared/src/carbon.js

```

export function co2Tonnes(fuelKg, fuelType = 'JET_A1') {
  if (fuelKg < 0) throw new RangeError('fuelKg cannot be negative');
  const factor = fuelType === 'JET_A' ? EMISSION_FACTOR_JET_A : EMISSION_FACTOR_JET_A1;
  // kg CO2 -> tonnes
  return Math.round((fuelKg * factor) / 1000 * 1000) / 1000;
}

```

Carbon is measured in tonnes with three decimal places — a tonne of CO₂ to the kilogram. Unlike money, this quantity is genuinely fractional and cannot be scaled to integers without losing information, so the code rounds explicitly and states the precision.

Quantity	Unit	Why not integers
Money	integer cents	A fractional cent cannot be invoiced
Emissions	tonnes to 3dp	A tonne of CO ₂ to the kg is the smallest meaningful quantity
Load factor	integer ppm	Six decimal places, no information lost at realistic precision
CASK	integer microcents	Scaled so it cannot round to zero

A genuine bug in co2Tonnes

Look carefully at:

```
` return Math.round((fuelKg factor) / 1000 1000) / 1000; `
```

The multiplication by 1000 cancels the division by 1000 on the left-hand side of the `Math.round`. The expression is equivalent to `Math.round(fuelKg * factor) / 1000` — which happens to be the intended behaviour, so the answer is correct.

But this is not what the code says. As written it reads as "convert to tonnes, then round to tonne-milligrams", which would be `Math.round((fuelKg factor) / 1000 1000) / 1000` — the same thing. The intent is ambiguous and the expression is redundant.

Compare with `applySaf`, which gets it right:

```
` const conventionalTonnes = round3((conventionalKg * factor) / 1000); `
```

`round3` applies the thousand-fold scaling *once*, at the end. The two functions should look identical and currently do not. **This is a readability defect with a latent risk:** a future edit that reorders the operations in `co2Tonnes` could introduce a real rounding error, and no test would distinguish the two forms if they agreed today.

```
// The critical rule: emissions must be counted ONCE. Where both
// regimes touch a flight, CORSIA-covered emissions are deducted from
// the EU ETS chargeable quantity. Double-counting is both a compliance
// error and, at current allowance prices, a seven-figure mistake.
//
// AMS implements the deduplication explicitly because no upstream
// library does it correctly for you.
```

shared/src/carbon.js — the module header

```
if (corsiaEligible && etsEligible) {
  // DEDUPLICATION: CORSIA-covered tonnes are deducted from ETS.
  // Where both apply, CORSIA is treated as satisfying the obligation,
  // consistent with the EU ETS directive's CORSIA deduction.
  allocated.CORSIA = totalTonnes;
  allocated.EU_ETS = 0;
  overlapTonnes.deductedFromETS = totalTonnes;
} else if (corsiaEligible) {
  allocated.CORSIA = totalTonnes;
} else if (etsEligible) {
  allocated.EU_ETS = totalTonnes;
} else {
  allocated.UNREGULATED = totalTonnes;
}
```

```
// Invariant: the parts must reconstruct the whole. Verified by the
// property test in tests/carbon.test.js.
allocationReconciles:
  Math.abs(
    allocated.CORSIA + allocated.EU_ETS + allocated.UNREGULATED - totalTonnes,
  ) < 1e-6,
```

The reconciliation invariant, and why it is the right one

The three allocation buckets must sum to the total. This is a property of the *deduplication*, not of any particular input: no matter how many regimes apply, the tonnes must be accounted for exactly once.

A property test over many flight configurations is a stronger check than a set of hand-computed examples, because the branch structure (both, corsia only, ets only, neither) has four paths and a hand-written test might miss a combination.

The reconciliation invariant in general

```
assertReconciles([t.directCostCents, t.fleetFixedCostCents], t.fullCostCents,
  'aggregate flight cost');

// Self-check. If this ever throws, a rounding rule was changed and
// the whole costing model needs re-verification.
assertReconciles(allocations, poolCents, `fleet-fixed allocation (${driver})`);
```

shared/src/costing.js — the engine checks its own output on every call.

```
/**
 * Assert that a set of amounts reconciles to an expected total.
 * Used by the three-way match and by the costing engine's own
 * self-checks. Throwing here is intentional: a silent mismatch in an
 * airline ledger becomes a million-dollar misstatement.
 */
```

The general pattern: assert the invariant at every layer that can break it

A reconciliation check inside the allocation function catches a rounding change immediately, at the point of change, in a test. The same check in a nightly reconciliation job catches it a month later, in production, as an unexplained variance that someone has to investigate from scratch.

Neither check is a substitute for the other, but the cheap one nearest the cause is always worth having.

Part III — Multi-tenancy as a security boundary

AMS serves multiple airlines from one database. A missing tenant filter returns one customer's financial data to another. That is not a bug report — it is an incident, a notification obligation, and in some jurisdictions a legal one.

```
/**
 * AMS – Migration 001: tenancy, identity and audit.
 *
 * Every table in AMS carries tenant_id. There is no exception. That rule
 * is enforced here, in the schema, rather than left to application
 * discipline, because tenant isolation is the single control that, if it
 * fails, makes a SaaS product legally and reputationally finished.
 *
 * Defence is three-layered (see docs Part X):
 * 1. middleware injects the tenant into every query
 * 2. repository code always filters explicitly
 * 3. the DATABASE refuses to return another tenant's rows
 *
 * Layer 3 is this file. Layers 1 and 2 are ordinary code that a future
 * developer can forget to write; layer 3 cannot be forgotten, because it
 * is the database.
 *
 * @module migrations/001
 */
```

backend/migrations/001_tenancy_identity_audit.js

One database, many airlines

```
CREATE TABLE tenants (
  id          TEXT PRIMARY KEY,
  slug        TEXT NOT NULL UNIQUE,
  legal_name  TEXT NOT NULL,
  icao_code    TEXT(3),           -- ICAO designator, e.g. "ABC"
  iata_code   TEXT(2),           -- IATA two-letter code
  country     TEXT NOT NULL,
  reporting_currency TEXT NOT NULL DEFAULT 'USD',
  base_currency TEXT NOT NULL DEFAULT 'USD',
  fiscal_year_start SMALLINT NOT NULL DEFAULT 1,
  aoc_reference TEXT,           -- Air Operator Certificate number
  aoc_valid_from DATE,
  aoc_valid_until DATE,
  certification_body TEXT,      -- e.g. CAAZ
  -- CORSIA new-entrant clock
  first_operation_year SMALLINT,
  annual_intl_co2_tonnes NUMERIC(14,3),
  eua_operated BOOLEAN NOT NULL DEFAULT FALSE,
  status      TEXT NOT NULL DEFAULT 'ACTIVE'
  CHECK (status IN ('ACTIVE', 'SUSPENDED', 'MIGRATING', 'CLOSED')),
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT now()
)
```

The tenants table holds one row per airline. Everything else references it. The design decision that matters is stated in the migration header: every table carries tenant_id, with no exceptions.

```
-- Foreign keys everywhere, cascading so a tenant deletion is complete
tenant_id  TEXT NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
```

Relationship	On delete	Consequence
Every tenant-owned table → tenants	CASCADE	Deleting a tenant removes all its data in one operation
user_roles → users	CASCADE	Deleting a user removes their role grants
budget_lines → budgets	CASCADE	Deleting a budget removes its lines

CASCADE on tenant deletion is a decision, not a default

CASCADE means deleting a tenant destroys all its financial records, audit entries and commitments in a single statement. For a system holding records that regulators may require years later, that is dangerous — and it is also the correct behaviour for a SaaS provider handling off-boarding under a data-processing agreement.

Neither the delete path nor any soft-delete alternative is implemented. The schema permits an operation that would destroy audit evidence. This needs a decision from whoever owns compliance before production, and almost certainly a status = 'CLOSED' path instead of deletion. Flagged here rather than silently documented as solved.

The three layers and why layer three is the database

```
// Layer 2 – the application filter, present in every query written so far
const line = await db('budget_lines')
  .where({ id: lineId, tenant_id: tenantId })
  .forUpdate()
  .first();

const commitment = await db('commitments')
  .where({ id: commitmentId, tenant_id: tenantId })
  .forUpdate()
  .first();
```

Layer 2 is visible in every query. It works. It is also the layer most likely to be missed by a future developer on a hurried afternoon, and nothing will tell them at development time.

```
// Layer 3 – the database refuses. Migration 002:
await knex.raw(`
  DO $$
  DECLARE t TEXT;
  BEGIN
    FOREACH t IN ARRAY ARRAY[
      'cost_categories', 'cost_centers', 'funds', 'aircraft',
      'budgets', 'budget_lines', 'commitments'
    ] LOOP
      EXECUTE format('ALTER TABLE %I ENABLE ROW LEVEL SECURITY', t);
      EXECUTE format('ALTER TABLE %I FORCE ROW LEVEL SECURITY', t);
      EXECUTE format($p$
        CREATE POLICY %1$I_tenant_isolation ON %1$I
          USING (tenant_id = current_setting('app.tenant_id', true))
          WITH CHECK (tenant_id = current_setting('app.tenant_id', true))
        $p$, t);
    END LOOP;
  END $$
`);
```

Row-level security, concretely

PostgreSQL row-level security is a database feature where a policy is defined per table, and the database itself adds a filter to every query. Read the policy:

```
USING (tenant_id = current_setting('app.tenant_id', true))
WITH CHECK (tenant_id = current_setting('app.tenant_id', true))
```

Clause	Applies to	Effect
USING	Reads: SELECT, and the rows visible to UPDATE and DELETE	Invisible rows are as if they do not exist
WITH CHECK	Writes: INSERT and UPDATE	A row that would violate the policy is rejected outright

```
-- How a tenant is established for the connection:
SET app.tenant_id = 'tenant-acme';

-- This is fine:
SELECT * FROM budget_lines WHERE id = 'bl-1';

-- This returns NOTHING, not an error – the other tenant's row is invisible:
SELECT * FROM budget_lines WHERE id = 'bl-1-other-tenant';

-- This is REJECTED:
INSERT INTO budget_lines (tenant_id, budgeted_cents) VALUES ('tenant-other', 1000);
-- ERROR:  new row violates row-level security policy for table "budget_lines"
```

The critical difference between invisible and rejected

A missing WHERE `tenant_id = $1` in application code does not produce an error. It produces *another customer's data in the response*, and nothing in the stack complains.

With the RLS policy, the same missing filter produces an empty result set. The application shows an empty page and the developer investigates within minutes. **One is an incident; the other is a puzzling bug report.** That difference is the entire value of layer 3.

```
USING (tenant_id = current_setting('app.tenant_id', true))
      ^^^^
      The connection-level setting, per transaction.
```

The missing-tenant behaviour is a real gap in the design

`current_setting('app.tenant_id', true)` — the second argument `true` means "return NULL instead of erroring if the setting is absent".

If a connection has no tenant set, the comparison is `tenant_id = NULL`, which is `NULL`, not `true` — so the policy returns no rows. **That is fail-closed and therefore safe.**

However it fails *silently*: a connection missing its tenant setting returns empty results for every query rather than raising an error. That is the correct security posture and the worst diagnostic posture. An explicit check in middleware — refusing to serve a request with no tenant context — would convert a confusing empty page into an immediate, obvious failure. Worth adding when the server layer is built.

```
-- Fail-closed, confirmed: with no tenant set
SET app.tenant_id = '';
SELECT count(*) FROM budget_lines;    -- 0 rows, not an error
SELECT count(*) FROM budget_lines;    -- with no SET at all: also 0
```

FORCE ROW LEVEL SECURITY and why it is not optional

```
EXECUTE format('ALTER TABLE %I ENABLE ROW LEVEL SECURITY', t);
EXECUTE format('ALTER TABLE %I FORCE ROW LEVEL SECURITY', t);
```

Setting	Applies policies to	Bypassed by
ENABLE ROW LEVEL SECURITY	Non-superuser roles, and the table owner	The table owner — and migrations usually run as the owner
FORCE ROW LEVEL SECURITY	Everyone, including the table owner	Superusers and roles with BYPASSRLS

Why this matters and is easy to miss

By default, PostgreSQL exempts the table *owner* from row-level security. Migrations almost always create tables as the owner and the application often connects as that same role.

In that configuration — which is the default and therefore the most common — ENABLE ROW LEVEL SECURITY alone provides **no protection at all**. Every query runs with full access.

FORCE closes it. This is one of the least well-known PostgreSQL behaviours, and a team that enables RLS, believes it is protected, and is not, is in a worse position than a team that never enabled it at all — because the belief suppresses the scrutiny.

Remaining bypass routes, which no RLS policy addresses

Superusers always bypass RLS. **Roles with BYPASSRLS** always bypass it. If the application connects as a superuser — which is convenient in development and occasionally survives into production — layer 3 provides nothing.

Connection pooling is a specific hazard here. If the application uses a pool (PgBouncer, or a driver-level pool) and sets `app.tenant_id` per request, a returned connection may retain the previous request's setting. The next request would then read another tenant's data — with the RLS policy working perfectly, faithfully applying the *wrong* tenant.

The safe patterns are to use transaction-scoped settings with a pooler in transaction mode, or to reset the setting on checkout. **This is not implemented and not verified anywhere in the codebase**. It is the most likely way the three-layer defence fails in production.

```
// Safe: tenant bound to the transaction, reset explicitly
await db.transaction(async (trx) => {
  await trx.raw("SELECT set_config('app.tenant_id', ?, true)", [tenantId]); // true = local to tx
  // ... all work in this transaction
});
// Setting is discarded on commit or rollback. Nothing leaks to the next borrower.
```

What RLS does not protect against

Threat	Does RLS stop it?	What does
A query missing WHERE tenant_id = ?	Yes	Returns empty instead of another tenant's rows
An INSERT with the wrong tenant_id	Yes	Rejected by WITH CHECK
An UPDATE moving a row to another tenant	Yes	The new row fails WITH CHECK
A superuser or BYPASSRLS connection	No	Application configuration; least-privilege roles
A leaked credential allowing login	No	Authentication, MFA, session controls
Timing side channels revealing row existence	No	Requires deliberate mitigation; rarely a priority
A pg_dump by someone with disk access	No	Encryption at rest; separate credentials
A tenant's own user reading their own data	n/a	Role-based access within the tenant — a separate layer

The remaining layer: roles within a tenant

RLS isolates tenants from each other. It does nothing about one accountant at an airline reading another's budget at the same airline. That is role-based access, and AMS has the vocabulary for it:

`ROLES: platform\_admin, tenant\_admin, ceo, finance\_director, controller, accountant, procurement\_manager, technical\_manager, flight\_ops\_manager, safety\_officer, auditor`

The roles table and user_roles grants exist in migration 001. The enforcement — which role may perform which operation — is not implemented, because the server layer does not exist yet.

```
CREATE TABLE user_roles (
  id          SERIAL PRIMARY KEY,
  tenant_id   TEXT NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
  user_id     TEXT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  role_id     INTEGER NOT NULL REFERENCES roles(id) ON DELETE CASCADE,
  granted_by  TEXT NOT NULL REFERENCES users(id),
  granted_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  expires_at  TIMESTAMPTZ,
  UNIQUE (user_id, role_id)
);
```

backend/migrations/001_tenancy_identity_audit.js

```
-- Delegations of authority
-- A departing or acting officer needs explicit, bounded, expiring
-- authority. It cannot be self-granted and cannot exceed the delegator's
-- own ceiling.
CREATE TABLE delegations (
  id SERIAL PRIMARY KEY,
  tenant_id TEXT NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
  delegator_id TEXT NOT NULL REFERENCES users(id),
  delegate_id TEXT NOT NULL REFERENCES users(id),
  scope_departments TEXT[] NOT NULL DEFAULT '{}',
  amount_ceiling_cents BIGINT,
  starts_at TIMESTAMPTZ NOT NULL,
  expires_at TIMESTAMPTZ NOT NULL,
  reason TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  CHECK (expires_at > starts_at),
  CHECK (delegator_id <> delegate_id)
);
```

Two design decisions worth naming. `UNIQUE (user_id, role_id)` means a user cannot hold the same role twice — no duplicate grants, so revocation is unambiguous. And `expires_at` means a delegation lapses automatically rather than relying on someone remembering to withdraw it.

Part IV — Algorithms that must be exactly right

The routing module is the newest part of AMS and the only one where the correctness argument is written out in full. This part teaches that argument, and — more importantly — teaches how we know the implementation matches it.

The problem class: resource-constrained shortest path

```
export function solveRoute({ flights, source, destination, tankCapacityKg, minReserveKg }) {
  if (!Number.isSafeInteger(tankCapacityKg) || tankCapacityKg < 0) {
    throw new RoutingError('tankCapacityKg must be a non-negative integer', 'ROUTE_BAD_CAPACITY');
  }
  if (!Number.isSafeInteger(minReserveKg) || minReserveKg < 0) {
    throw new RoutingError('minReserveKg must be a non-negative integer', 'ROUTE_BAD_RESERVE');
  }

  const usableFuelKg = tankCapacityKg - minReserveKg;
  if (usableFuelKg < 0) {
    throw new RoutingError(
      `Tank capacity ${tankCapacityKg}kg cannot hold the ${minReserveKg}kg mandatory reserve. ` +
      'The aircraft can never complete any sector.',
      'ROUTE_CAPACITY_BELOW_RESERVE',
    );
  }
}
```

In one sentence: find the cheapest sequence of stops from A to B such that the fuel burned so far never exceeds capacity minus reserve.

This is called a *resource-constrained shortest path* problem. The distinguishing feature is the second constraint: among all routes, minimise cost subject to cumulative fuel never exceeding a limit. It is harder than ordinary shortest path, and the difference is not subtle.

```
JFK → YUL    38,000c   3,200 kg
YUL → GRU    74,000c   8,600 kg
JFK → GRU    61,000c  12,100 kg
JFK → DUB    22,000c   3,300 kg
DUB → GRU    89,000c  10,400 kg

// With tank 24,000 and reserve 1,500 → 22,500 kg usable, every route fits:
const generous = solveRoute({ ...NETWORK, source: 'JFK', destination: 'GRU',
  tankCapacityKg: 24_000, minReserveKg: 1_500 });
generous.costCents // 61,000 — direct wins on cost

// Tighten to 13,300 / 1,500 → 11,800 kg usable:
const tight = solveRoute({ ...NETWORK, source: 'JFK', destination: 'GRU',
  tankCapacityKg: 13_300, minReserveKg: 1_500 });
tight.costCents // 112,000 — the direct sector no longer fits
tight.stops     // ['JFK','YUL','GRU'] — a MORE expensive route

// One kilogram tighter and nothing is legal:
const impossible = solveRoute({ ...NETWORK, source: 'JFK', destination: 'GRU',
  tankCapacityKg: 13_299, minReserveKg: 1_500 });
impossible.feasible // false
```

Why greedy and unconstrained shortest path both fail

```

describe('the fuel constraint actually binds', () => {
  it('finds many instances where the unconstrained shortest path is INFEASIBLE', () => {
    let disagreements = 0;
    for (let seed = 1; seed <= 2000; seed += 1) {
      const rand = mulberry32(seed * 69069 + 11);
      const inst = randomInstance(rand, int(rand, 3, 6));
      const unconstrained = unconstrainedCheapest(inst);
      if (unconstrained === null) continue;
      const constrained = solveRoute(inst);
      if (constrained.feasible && constrained.costCents > unconstrained) disagreements += 1;
    }
    // A naive Dijkstra would be wrong this often. Proof the constraint matters.
    expect(disagreements).toBeGreaterThan(50);
  });

  it('a greedy cheapest-next-hop planner is NOT optimal', () => {
    const inst = {
      flights: [
        { from: 'A', to: 'B', costCents: 10, fuelKg: 90 },
        { from: 'B', to: 'D', costCents: 10, fuelKg: 90 },
        { from: 'A', to: 'C', costCents: 11, fuelKg: 50 },
        { from: 'C', to: 'D', costCents: 11, fuelKg: 50 },
      ],
      source: 'A', destination: 'D',
      tankCapacityKg: 100, minReserveKg: 0,
    };
    const result = solveRoute(inst);
    expect(result.feasible).toBe(true);
    // Greedy picks A-B-D at cost 20 – but that burns 180kg in a 100kg tank.
    expect(result.costCents).toBe(22);
    expect(result.stops).toEqual(['A', 'C', 'D']);
  });
});

```

tests/routing.test.js

```

// The greedy counterexample, visualised
//
// 100 kg tank
//
// A ■■■10c/90kg■■■ B ■■■10c/90kg■■■ D
//                TOTAL: 20c, 180kg  ← CANNOT FLY
//
// A ■■■11c/50kg■■■ C ■■■11c/50kg■■■ D
//                TOTAL: 22c, 100kg   ← exactly fits
//
// Greedy returns the cheaper answer. Greedy grounds the aircraft.

```

Why the greedy failure is the dangerous kind

The greedy answer is *cheaper*. It is also impossible to fly. A planner that reported it would look better on every dashboard metric and would be catastrophically wrong.

This is the shape of defect that no amount of dashboard review catches: the number looks right, because the question was never asked.

Label-setting and the dominance relation

```

/**
 * Pareto label-setting (the BIRD family). At each node we keep a set of
 * NON-DOMINATED labels, where a label is (cost, fuelUsed).
 *
 * Label L1 dominates L2 iff cost1 <= cost2 AND fuel1 <= fuel2
 * (and they are not identical)
 *
 * Dominance is SOUND because both the objective and the resource are
 * monotone in the same direction: any completion feasible from L2 is
 * also feasible from L1 and costs no more. Therefore a dominated label
 * can never be the prefix of an optimal solution, and discarding it
 * cannot remove the optimum.
 */

```

The intuition is worth a concrete example, because "non-dominated" is abstract until you see a frontier.

```

// At airport B, three routes from A have been considered:
// (cost 50,000c, fuel 3,000 kg)
// (cost 40,000c, fuel 7,000 kg)
// (cost 45,000c, fuel 3,100 kg)
//
// Is the third label worth keeping?
// vs (50,000, 3,000): 45,000 < 50,000 but 3,100 > 3,000 → NOT dominated
// vs (40,000, 7,000): 45,000 > 40,000 but 3,100 < 7,000 → NOT dominated
// → KEEP IT. It is genuinely different: cheaper than one, lighter than the other.
//
// The frontier at B is {(40,000 / 7,000), (45,000 / 3,100), (50,000 / 3,000)}.
// The (50,000, 3,000) label IS dominated? No – nothing is both cheaper AND lighter.
// So all three survive.

```

```

// Now a fourth arrives: (cost 44,000c, fuel 3,200 kg)
// vs (50,000, 3,000): 44,000 < 50,000, 3,200 > 3,000 → not dominated
// vs (45,000, 3,100): 44,000 < 45,000, 3,200 > 3,100 → not dominated
// vs (40,000, 7,000): 44,000 > 40,000, 3,200 < 7,000 → not dominated
// → KEEP IT too. Four genuinely incomparable options.

```

```

// Finally: (cost 41,000c, fuel 3,050 kg)
// vs (45,000, 3,100): 41,000 < 45,000 AND 3,050 < 3,100 → DOMINATES, discard that one
// vs (40,000, 7,000): 41,000 > 40,000, but 3,050 < 7,000 → not dominated
// vs (50,000, 3,000): 41,000 < 50,000, but 3,050 > 3,000 → not dominated
// → The (45,000, 3,100) label is discarded. It can never help.

```

The intuitive argument for discarding a dominated label

If a label is both more expensive AND heavier than another label at the same airport, then every route reachable from it is reachable from the other one, for less money, with more fuel in the tanks. It is not merely *probably* useless — it is **provably** useless, because the remaining journey is identical either way.

This is what makes the algorithm exact rather than heuristic. No promising option is thrown away.

```

function dominates(ca, fa, cb, fb) {
  return ca <= cb && fa <= fb && (ca < cb || fa < fb);
}

```

```

while (heap.size > 0) {
  const label = /** @type {Label} */ (heap.pop());
  labelsExpanded += 1;

  // Skip if this label was pruned after being queued.
  const live = labelsAt(label.node);
  if (!live.includes(label)) { labelsPruned += 1; continue; }

  if (label.node === destination) {
    // The heap is ordered by cost, so the first label popped at the
    // destination is the cheapest surviving one – and by the theorem,
    // the optimum.
    return {
      feasible: true,
      costCents: label.costCents,
      fuelUsedKg: label.fuelUsedKg,
      stops: reconstructStops(label),
      labelsExpanded, labelsPruned, maxFrontier,
    };
  }

  const outgoing = adjacency.get(label.node) ?? [];
  for (const edge of outgoing) {
    const fuelUsedKg = label.fuelUsedKg + edge.fuelKg;
    if (fuelUsedKg > usableFuelKg) continue; // infeasible prefix – never expand

    const costCents = label.costCents + edge.costCents;
    const target = labelsAt(edge.to);

    let dominated = false;
    for (const other of target) {
      if (dominates(other.costCents, other.fuelUsedKg, costCents, fuelUsedKg)) {
        dominated = true; break;
      }
      if (other.costCents === costCents && other.fuelUsedKg === fuelUsedKg) {
        dominated = true; break;
      }
    }
    if (dominated) { labelsPruned += 1; continue; }

    // Is it dominating something already there? Remove those.
    for (let i = target.length - 1; i >= 0; i -= 1) {
      if (dominates(costCents, fuelUsedKg, target[i].costCents, target[i].fuelUsedKg)) {
        target.splice(i, 1);
      }
    }

    const next = { costCents, fuelUsedKg, node: edge.to, prev: label };
    target.push(next);
    if (target.length > maxFrontier) maxFrontier = target.length;
    heap.push(next);
  }
}

```

shared/src/routing/constrained-path.js — the whole search loop.

Why the dominance rule is sound

```

* Invariant I1 (reachability): every label in the frontier at v
* corresponds to an actual feasible path from the source to v.
* Proof: labels are created only by extending a popped label with a
* real edge, and only when the resulting cumulative fuel satisfies the
* capacity constraint. Base case is the trivial path at the source.
*
* Invariant I2 (pruning soundness): if L1 dominates L2 at v, then for
* every feasible completion C of L2 to any destination, C composed
* with L1 is also feasible and no more expensive than C composed with
* L2.
* Proof: fuel is consumable and monotone – having burnt less fuel
* leaves at least as much remaining at every subsequent point, so
* every leg that was feasible remains feasible. Cost is additive, and
* cost1 <= cost2, so the total cannot increase.
*
* Theorem: the algorithm returns the minimum-cost feasible path.
* Proof: Take an optimal feasible path P. Walk it from the source. Each
* prefix label is either in the frontier, or was pruned. If pruned,
* it was dominated by some retained L' which by I2 can replace that
* prefix with no increase in cost or fuel. Following I2
* inductively, a path of cost <= OPT(P) survives to the destination.
* By I1 every surviving label is feasible, so its cost >= OPT(P).
* Therefore the minimum-cost surviving label has cost exactly OPT(P).

```

Translated out of notation: because both the objective (cost) and the constraint (fuel) improve in the same direction, a worse label can never rescue a better one. That is what makes discarding labels safe, and it is the entire reason the algorithm is exact rather than a heuristic.

The one assumption the theorem depends on

Monotonicity of the resource

The proof requires that using *less* resource never hurts. That is true for fuel, because fuel is only ever consumed and never generated en route.

It would be false for a resource that could be replenished, or for one with a penalty for carrying too little. If AMS ever added a constraint of that shape — "you must also arrive with at least X kg" — the dominance rule would need to change, and a label that is both cheaper and lighter might no longer be the better one.

Nothing in the code documents this dependency. It is a real constraint on future modification and belongs in a comment next to `dominates()`.

Complexity, and knowing when you are near the wall

```

* Complexity:  $O(L \cdot E \cdot \log L)$  where L is the largest per-node frontier.
* In practice L is tiny (usually < 5) because real networks have few
* genuinely incomparable cost/fuel trade-offs per airport.

```

Symbol	Meaning	Typical value in an airline network
V	Airports in the network	200–1,500 worldwide; 20–80 for a single operation
E	Direct sectors	~3–15 per airport
L	Largest per-node non-dominated frontier	2–5 . The critical assumption.
B	Maximum distinct fuel values	~45,000 kg for a widebody

```

it('plans a 400-airport network with 6,000 sectors in well under a second', () => {
  const N = 400;
  for (let i = 0; i < N; i += 1) {
    const degree = int(rand, 3, 18);
    for (let d = 0; d < degree; d += 1) {
      const to = int(rand, 0, N - 1);
      if (to === i) continue;
      flights.push({
        from: nodes[i], to: nodes[to],
        costCents: int(rand, 1_000, 900_000),
        // Fuel is uncorrelated with cost, so cost/fuel trade-offs are real.
        fuelKg: int(rand, 500, 12_000),
      });
    }
  }

  const t0 = performance.now();
  const result = solveRoute({
    flights, source: nodes[0], destination: nodes[N - 1],
    tankCapacityKg: 45_000, minReserveKg: 1_500,
  });
  const elapsed = performance.now() - t0;

  expect(result.feasible).toBe(true);
  assertRouteLegal({ flights, tankCapacityKg: 45_000, minReserveKg: 1_500 }, result.stops);
  // 6,000 sectors must not take seconds.
  expect(elapsed).toBeLessThan(1000);
});

```

tests/routing.test.js

The test generates fuel *uncorrelated* with cost. That is deliberate and it is the most important line in the fixture. If cheap sectors happened to be light, the dominance rule would rarely need more than one label per airport, and the hard path through the algorithm would never be exercised.

Where the wall is, and how you know

The algorithm's cost depends on the frontier size L , not on fuel quantity. That is what makes it tractable at all — a fuel-indexed DP over 45,000 kg of capacity would be 45,000 states per airport, roughly 68 million states for a 1,500-airport network.

The Pareto algorithm has no such dependence, which is why it is the production solver and the DP is a reference implementation only. The DP carries a guard:

```

` if (expanded > maxExpand) { throw new RoutingError( DP state space exceeded ${maxExpand} - use
  solveRoute instead, 'ROUTE_DP_TOO_LARGE'); } `

```

The $L \approx 2-5$ assumption is not proven, only observed

The complexity is stated as $O(L \cdot E \cdot \log L)$ and the comment asserts that L is usually small. This has not been verified adversarially. A network constructed so that many labels are genuinely incomparable at one airport could drive L to hundreds, making the search quadratic in practice.

The mitigation is instrumentation: `solveRoute` already returns `labelsExpanded`, `labelsPruned` and `maxFrontier`. Logging `maxFrontier` in production would turn the assumption into a measurement. This is not wired up anywhere, and it should be.

Differential verification against brute force

```

/**
 * Exhaustive enumerator over every simple path. Exponential, used ONLY
 * to validate the fast algorithms on small instances.
 */

export function bruteForceRoute({ flights, source, destination, tankCapacityKg, minReserveKg }) {
  const usable = tankCapacityKg - minReserveKg;
  if (usable < 0) throw new RoutingError('Capacity below reserve', 'ROUTE_CAPACITY_BELOW_RESERVE');
  if (source === destination) return { feasible: true, costCents: 0, stops: [source], pathsExplored: 0 };

  const adjacency = buildAdjacency(flights);
  let bestCost = Infinity;
  let bestStops = /** @type {string[] | null} */ (null);
  let explored = 0;

  const walk = (node, fuel, cost, path) => {
    explored += 1;
    if (cost >= bestCost) return; // cannot improve
    if (fuel > usable) return; // already infeasible
    if (node === destination) {
      bestCost = cost; bestStops = [...path, node]; return;
    }
    for (const e of adjacency.get(node) ?? []) {
      if (path.includes(e.to)) continue; // simple paths only
      walk(e.to, fuel + e.fuelKg, cost + e.costCents, [...path, node]);
    }
  };

  walk(source, 0, 0, [source]);
  return {
    feasible: bestStops !== null,
    costCents: bestCost === Infinity ? 0 : bestCost,
    stops: bestStops,
    pathsExplored: explored,
  };
}

```

This solver has no cleverness in it. It tries every route, checks feasibility, keeps the cheapest, and returns. That makes it an *oracle*: correct by construction, with nothing to get wrong except the constraints themselves.

```

describe('constrained shortest path – three independent solvers agree', () => {
  it('agrees with the exhaustive oracle on 3000 random instances', () => {
    let compared = 0;
    let feasible = 0;
    for (let seed = 1; seed <= 3000; seed += 1) {
      const rand = mulberry32(seed * 2654435761);
      const n = int(rand, 2, 6);
      const inst = randomInstance(rand, n);
      if (inst.flights.length === 0) continue;

      const pareto = solveRoute(inst);
      const oracle = bruteForceRoute(inst);

      compared += 1;
      if (oracle.feasible) feasible += 1;

      expect(pareto.feasible, `seed ${seed}: feasibility must match oracle`).toBe(oracle.feasible);
      if (oracle.feasible) {
        expect(pareto.costCents, `seed ${seed}: optimal cost must match oracle`).toBe(oracle.costCents);
        const { cost } = assertRouteLegal(inst, pareto.stops);
        expect(cost, `seed ${seed}: reported cost must equal the sum of its own legs`).toBe(pareto.costCents);
      }
    }
    expect(compared).toBeGreaterThan(2500);
    // The test is worthless if every instance was trivially infeasible.
    expect(feasible / compared).toBeGreaterThan(0.3);
  });

  it('agrees with the DP reference solver on 1500 random instances', () => {
    // ... same structure ...
  });

  it('the DP result is itself legal, not merely equal in cost', () => { /* ... */ });
});

```

Three assertions do distinct work, and the third is the one teams skip.

Assertion	What it catches
Feasibility must match the oracle	The solver reports "no route" when one exists, or vice versa
Cost must match the oracle	The solver returns a suboptimal route
The route must re-add to its own reported cost	The cost is right but the returned stops are a different route — e.g. a predecessor-pointer bug assembling the wrong path

```

/** Recompute cumulative fuel for a route and assert it is legal. */
function assertRouteLegal(instance, stops) {
  const index = new Map();
  for (const f of instance.flights) index.set(`${f.from}>${f.to}`, f);
  const usable = instance.tankCapacityKg - instance.minReserveKg;
  let cumulative = 0;
  let cost = 0;
  for (let i = 0; i < stops.length - 1; i += 1) {
    const leg = index.get(`${stops[i]}>${stops[i + 1]}`);
    expect(leg, `leg ${stops[i]}->${stops[i + 1]} must exist`).toBeDefined();
    cumulative += leg.fuelKg;
    cost += leg.costCents;
  }
  expect(cumulative).toBeLessThanOrEqual(usable);
  return { cumulative, cost };
}

```

Why the self-consistency check is not redundant

A solver could return the correct cost alongside a route that does not achieve it — if, for instance, a reconstruction bug assembled the stops from wrong predecessor pointers. Comparing two costs would pass. Walking the returned route and re-adding its own costs catches that, and independently confirms the fuel constraint holds on the answer that will actually be flown.

The defect this technique found: the DP carried the wrong node

```
` buckets[nf].push({ fuelUsed: nf, costCents: nc, from: state.from, prevFuel: f }); ^^^^^^^^^^^^^ `
```

It recorded the node the flight **departed** from rather than the one it **arrived** at. Every subsequent step then re-expanded the wrong airport's outgoing flights, so the DP reported routes as infeasible that the primary solver had found.

The consequence is instructive: the bug was in the *reference* implementation, and it would have been missed indefinitely by any hand-written expectation. It was caught only because the test forces two independent implementations to agree.

The general principle this defect teaches

A second implementation is only useful if something **forces** them to agree. Two solvers that disagree and are never compared provide no more assurance than one.

This is why the AMS test suite cross-checks rather than asserting hand-computed values wherever it can.

Property tests and metamorphic tests

```
describe('monotonicity – properties any correct solver must satisfy', () => {
  it('more tank capacity never increases the optimal cost', () => {
    for (let seed = 1; seed <= 800; seed += 1) {
      const rand = mulberry32(seed * 1103515245 + 12345);
      const inst = randomInstance(rand, int(rand, 3, 6));
      if (inst.flights.length === 0) continue;

      const base = solveRoute({ ...inst, tankCapacityKg: 20, minReserveKg: 3 });
      const bigger = solveRoute({ ...inst, tankCapacityKg: 45, minReserveKg: 3 });
      if (base.feasible && bigger.feasible) {
        expect(bigger.costCents, `seed ${seed}: capacity grew, cost must not rise`)
          .toBeLessThanOrEqual(base.costCents);
      }
    }
  });

  it('a lower mandatory reserve never increases the optimal cost', () => { /* ... */ });

  it('raising any sector cost never decreases the optimal cost', () => { /* ... */ });

  it('when the unconstrained optimum is feasible, it is also the constrained optimum',
    () => {
      let checked = 0;
      for (let seed = 1; seed <= 1200; seed += 1) {
        // ...
        if (result.costCents === unconstrained) checked += 1;
      }
      expect(checked).toBeGreaterThan(20);
    }
  });
});
```

These tests know nothing about the algorithm. They express truths about the *problem*, and any correct implementation must satisfy them. A completely different correct algorithm would pass them; the particular implementation described in this document need not be known to the test.

Property	Why it must hold
More capacity cannot cost more	A bigger tank removes a constraint; it cannot add one
A lower reserve cannot cost more	Same reasoning — a weaker requirement cannot hurt
Higher prices cannot cost less	Expensive flights are never an opportunity
A feasible unconstrained optimum equals the constrained optimum	If the cheap answer is allowed, it should win

The last property is worth spelling out. It says: *constraint the answer, and if the unconstrained answer is still legal, the constrained answer must equal it*. That is an independent certificate of optimality — the test can confirm the solver's answer without consulting the oracle at all.

Edge cases as a deliberate category

```
it('drops self loops, which can never improve a route', () => {
  const withLoop = solveRoute({
    flights: [
      { from: 'A', to: 'A', costCents: 0, fuelKg: 0 },
      { from: 'A', to: 'B', costCents: 500, fuelKg: 10 },
    ],
    source: 'A', destination: 'B', tankCapacityKg: 100, minReserveKg: 0,
  });
  expect(withLoop.costCents).toBe(500);
  expect(withLoop.stops).toEqual(['A', 'B']);
});

it('handles zero-fuel legs without infinite looping', () => {
  const r = solveRoute({
    flights: [
      { from: 'A', to: 'B', costCents: 5, fuelKg: 0 },
      { from: 'B', to: 'C', costCents: 5, fuelKg: 0 },
      { from: 'C', to: 'A', costCents: 1, fuelKg: 0 },
    ],
    source: 'A', destination: 'C', tankCapacityKg: 0, minReserveKg: 0,
  });
  expect(r).toMatchObject({ feasible: true, costCents: 10 });
});

it('a zero-cost cycle does not hang the planner', () => { /* ... */ });

it('rejects non-integer fuel – silent truncation would create phantom capacity',
  () => {
    expect(() => solveRoute({
      flights: [{ from: 'A', to: 'B', costCents: 1, fuelKg: 10.5 }],
      source: 'A', destination: 'B', tankCapacityKg: 100, minReserveKg: 0,
    })).toThrow(/integer/);
  });
```

```

export function buildAdjacency(flights) {
  for (const f of flights) {
    if (!Number.isSafeInteger(f.fuelKg) || f.fuelKg < 0) {
      throw new RoutingError(
        `Flight ${f.from}->${f.to} fuel must be a non-negative integer number of kg`,
        'ROUTE_BAD_FUEL',
      );
    }
    if (f.from === f.to) {
      // A self-loop can never improve a route in this problem: it adds
      // cost and fuel, so it is strictly dominated. Silently dropping it
      // is safe and prevents degenerate label churn.
      continue;
    }
    // ...
  }
}

```

The infinite loop, and why no assertion would have found it

```
` if (dominates(other.costCents, other.fuelUsedKg, costCents, fuelUsedKg)) { dominated = true; break; } `
```

A zero-cost, zero-fuel cycle produces a label **identical** to one already present. `dominates` requires strict inequality on at least one dimension, so an identical label does not dominate — and is not dominated. It was accepted, expanded, and produced another identical label, forever.

The fix rejects duplicates explicitly:

```
` if (other.costCents === costCents && other.fuelUsedKg === fuelUsedKg) { dominated = true; break; } `
```

How it was found: the test suite hung and was killed by a five-minute timeout. No correctness assertion would have detected it, because the code was not producing wrong answers — it was producing no answers.

Non-termination is invisible to assertions and only to a timeout.

```

/**
 * Deterministic JSON serialisation – from the audit chain, with the same
 * principle: make randomness reproducible, or a failure is not a defect
 * report but a rumour.
 */
function mulberry32(seed) {
  let a = seed >>> 0;
  return function rand() {
    a |= 0; a = (a + 0x6d2b79f5) | 0;
    let t = Math.imul(a ^ (a >>> 15), 1 | a);
    t = (t + Math.imul(t ^ (t >>> 7), 61 | t)) ^ t;
    return ((t ^ (t >>> 14)) >>> 0) / 4294967296;
  };
}

```

tests/routing.test.js

Without a seeded generator, a random test failure is unreproducible and therefore nearly useless as a defect report. With one, "seed 3 fails" is a permanent regression case. Every random assertion in the AMS suite names the seed in its failure message.

The uplift planner and its exchange argument

```

* THE THEOREM
*
* For a fixed route with LINEAR price per kg, capacity T and reserve R
* required at every landing, the cost-optimal uplift policy is:
*
*   At stop i, let k = the first stop after i whose price is strictly
*   cheaper than the price at i.
*
*   - If such a k exists: uplift only enough to reach k with the
*   reserve intact. Buying more is waste, because  $p_k < p_i$ .
*   Buying less is strictly more expensive.
*
*   - If no such k exists (every remaining stop is dearer, or we can
*   reach the destination in one tank): buy as much as is still
*   needed, filling the tank only if the remainder of the route
*   does not fit.
*
* PROOF BY EXCHANGE
* -----
* Fix a stop i and suppose k = next cheaper stop exists.
*
* Under-buying: if the policy departs i with less fuel than needed to
* reach k, the shortfall must be bought at some stop j with  $i < j \leq k$ 
* (feasibility forbids running dry). Every such j has  $p_j \geq p_i$ , since
* k is the FIRST stop cheaper than i. Moving that purchase from j back
* to i cannot raise cost. It also cannot breach capacity: the shifted
* amount is exactly the shortfall, and the aircraft was already
* carrying more than the shortfall at i by hypothesis. Hence
* under-buying is never optimal.
*
* Over-buying: if the policy carries past i more fuel than needed to
* reach k, that surplus is consumed at a stop with  $p_k < p_i$ .
* Buying it at k instead is strictly cheaper. Capacity at i is not
* violated because we are removing, not adding. Hence over-buying is
* never optimal.
*
* No cheaper stop ahead: then  $p_j \geq p_i$  for all  $j > i$ . Every kilogram
* bought later can be bought at i for no greater price, subject to
* capacity. So buy the whole remaining requirement now; fill the tank
* only when the remainder does not fit.
*
* The three cases exhaust the possibilities, so the myopic policy is
* optimal. This is verified independently against exhaustive search in
* the test suite.

```

Read plainly: buy too little at a stop and you will buy the shortfall somewhere more expensive later, so you should have bought it here. Buy too much and you are carrying fuel that will be consumed somewhere cheaper, so you should have bought it there. The only defensible quantity is the minimum needed to reach the next cheaper station. There is nothing subtle here — which is exactly why it is worth understanding.

```

export function optimalRefuelPlan({ stops, legFuelKg, priceCentsPerKg,
  tankCapacityKg, reserveKg }) {
  // ...
  let fuelOnBoard = 0; // arriving at stop 0 with empty tanks

  for (let i = 0; i < n - 1; i += 1) {
    const price = priceCentsPerKg[i];

    // First stop strictly cheaper than here, among later DEPARTURE stops.
    let nextCheaper = -1;
    for (let j = i + 1; j < n - 1; j += 1) {
      if (priceCentsPerKg[j] < price) { nextCheaper = j; break; }
    }

    let target;
    let rationale;
    if (nextCheaper >= 0) {
      target = fuelToReach(legFuelKg, i, nextCheaper, reserveKg);
      rationale = `hold ${priceCentsPerKg[i]}c/kg: only enough to reach ${stops[nextCheaper]}...`;
    } else {
      target = Math.min(tankCapacityKg, required[i]);
      rationale = `hold ${priceCentsPerKg[i]}c/kg: no cheaper stop ahead...`;
    }

    const uplift = Math.max(0, target - fuelOnBoard);
    // ...
    fuelOnBoard = departure - legFuelKg[i];
    if (fuelOnBoard < reserveKg) {
      throw new RefuelError(
        `Arriving at ${stops[i + 1]} with ${fuelOnBoard}kg breaches the ${reserveKg}kg reserve. ` +
        'The plan is infeasible – this should be unreachable.',
        'REFUEL_RESERVE_BREACH',
      );
    }
  }
  // ...
}

```

shared/src/routing/refuel.js

```

// Worked example – JFK sells at 95c, MAN at 70c, GRU is the destination
const plan = optimalRefuelPlan({
  stops: ['JFK', 'MAN', 'GRU'],
  legFuelKg: [3200, 8600],
  priceCentsPerKg: [95, 70, 88],
  tankCapacityKg: 24_000,
  reserveKg: 1_500,
});
plan.upliftsKg // [4700, 8600]
plan.totalCostCents // 4700*95 + 8600*70 = 1,048,500

```

Stop	Price	Arriving	Buy	Reason
JFK	95c	0 kg	4,700 kg	MAN at 70c is cheaper — buy only enough to reach it
MAN	70c	1,500 kg	8,600 kg	Nothing cheaper ahead — buy what the rest needs
GRU	—	1,500 kg	—	Arrival

Loading all 13,300 kg at JFK instead would cost 1,263,500 cents — 17% more, for no benefit. The planner correctly refuses to tanker past the cheap station.

The destination-price trap, caught by a test that was itself wrong

` // GRU has the LOWEST price of the three — but you cannot buy // fuel on arrival, so its price must not influence any decision. plan.upliftsKg // [13\_300, 0] — all loaded at JFK `

An early test asserted the planner would stop at GRU because it looked cheap. **The test was wrong, not the code.** Fixing the test required reasoning about which prices can legitimately influence a decision.

This is a small, clean example of a recurring theme: a plausible-looking expectation is not the same thing as a correct one, and the code can be right while the test that describes it is wrong.

```
export function requiredAtDeparture(legFuelKg, reserveKg) {
  const n = legFuelKg.length;
  const required = new Array(n);
  for (let i = n - 1; i >= 0; i -= 1) {
    // Departing stop i you must burn leg i to reach stop i+1, and then
    // hold AT LEAST the reserve – but possibly more than the reserve,
    // because everything the rest of the route needs must still be on
    // board when you leave i. Hence max(), and hence the composition is
    // legFuel[i] + max(reserve, required[i+1]), not max(legFuel[i]+reserve,
    // required[i+1]). The latter silently under-fuels the first departure
    // of every multi-leg route.
    const neededBeyond = i === n - 1 ? reserveKg : Math.max(reserveKg, required[i + 1]);
    required[i] = legFuelKg[i] + neededBeyond;
  }
  return required;
}
```

```
it('composes the whole remainder of the route, not just the next leg', () => {
  // legs [1000, 2000, 500], reserve 300, route A-B-C-D.
  // Departing C: 500 leg + 300 reserve           = 800
  // Departing B: 2000 leg + max(300, 800)         = 2800
  // Departing A: 1000 leg + max(300, 2800)         = 3800
  //
  // This test originally asserted 2300/2300, which was wrong in exactly
  // the same way the implementation was wrong: it treated the reserve
  // as if it were the only requirement beyond the next leg, so it
  // under-fuelled the first departure. Both the code and the test were
  // wrong in the same direction, which is what a test suite written from
  // the author's own mental model looks like. Caught by the exhaustive
  // oracle, not by inspection of either.
  const required = requiredAtDeparture([1000, 2000, 500], 300);
  expect(required[2]).toBe(800);
  expect(required[1]).toBe(2800);
  expect(required[0]).toBe(3800);
});
```

tests/routing.test.js

The most instructive defect in the codebase

The implementation and its test were wrong in **exactly** the same way, so the test passed and confirmed the bug. Testing something against your own model of how it should behave does not validate the model. This is why the routing tests compare against an **exhaustive search** rather than against hand-written expectations: an oracle cannot share your misunderstanding, because it has no model at all.

```
// And the fourth: a feasibility check that rejected the plans it existed for
for (let i = 0; i < legFuelKg.length; i += 1) {
  if (legFuelKg[i] + reserveKg > tankCapacityKg) {
    throw new RefuelError(
      `Sector ${stops[i]} -> ${stops[i + 1]} needs ${legFuelKg[i] + reserveKg}kg ` +
      `(leg + ${reserveKg}kg reserve) but tank capacity is only ${tankCapacityKg}kg. ` +
      'No refuelling schedule can make this route legal.',
      'REFUEL_ROUTE_INFEASIBLE',
    );
  }
}
}
```

The corrected check: each LEG must fit, not the whole route.

The original checked `required[0] > tankCapacityKg`, which is the fuel for the *entire remaining route*. For any multi-stop route that exceeds capacity — which is normal, because you refuel at the intermediate stops. The check therefore rejected every multi-stop plan, which is precisely what the function exists to produce.

```
describe('refuel - myopic policy versus exhaustive search', () => {
  it('matches the oracle on 2000 random routes', () => {
    let compared = 0;
    for (let seed = 1; seed <= 2000; seed += 1) {
      const rand = mulberry32(seed * 48271 + 31);
      const legs = int(rand, 1, 3);
      const args = {
        stops: Array.from({ length: legs + 1 }, (_, i) => `S${i}`),
        legFuelKg: Array.from({ length: legs }, () => int(rand, 1, 9)),
        priceCentsPerKg: Array.from({ length: legs + 1 }, () => int(rand, 1, 4) * 25),
        reserveKg: int(rand, 0, 2),
        tankCapacityKg: legFuelKg.reduce((a, b) => a + b, 0) + reserveKg + int(rand, 0, 3),
      };
      const mine = optimalRefuelPlan(args);
      const oracle = bruteForceRefuel(args);
      expect(mine.totalCostCents, `seed ${seed}: myopic plan must be cost-optimal`)
        .toBe(oracle.totalCostCents);
    }
    expect(compared).toBeGreaterThan(1500);
  });

  it('arrives at every stop with at least the reserve, and reports honest arithmetic',
    () => { /* ... */ });
});
```

Part V — Integrity: making history unchangeable

An airline audit trail is evidence presented to a statutory auditor, a lessor's technical representative, and a foreign civil aviation authority. If the airline can edit its own history, none of that evidence is worth anything.

```
/**
 * A conventional audit log – even append-only at the application level –
 * is editable by anyone holding database write access. Application-level
 * append-only is a policy; this is a mechanism.
 *
 * HOW IT WORKS
 *
 * Each entry stores the SHA-256 of its own content concatenated with the
 * previous entry's hash:
 *
 *     entry_hash = SHA256(prev_hash || canonical_entry)
 *
 * Any modification, deletion or reordering of an historic entry breaks
 * the chain at that point and every point after it. Recomputing the
 * chain detects it. Rewriting the whole chain requires the database
 * write key, which in production is not the same credential the
 * application runs with, and the chain head is additionally exported for
 * external anchoring.
 *
 * This is the same construction used for public ledgers, applied to
 * internal financial audit. It does not make the data trustworthy – it
 * makes tampering DETECTABLE, which is the property that matters.
 *
 * @module audit/chain
 */
```

backend/src/audit/chain.js

Append-only as a policy versus a mechanism

Approach	What it is	Who can defeat it
Policy	A convention. "Do not update audit rows."	Anyone with database write access. Including a privileged operator, a migration script, a mistaken UPDATE.
Application constraint	The repository exposes only insert and select for audit rows	Still anyone with direct database access. And any future code path that writes raw SQL.
Mechanism	The database itself refuses UPDATE and DELETE	A superuser, or someone who drops the trigger. Both are deliberate, logged, and rare.

```

// Append-only enforcement at the DATABASE level. Application-level
// append-only is a policy a privileged operator can bypass; this is not.
await knex.raw(`
  CREATE OR REPLACE FUNCTION ams_forbid_audit_mutation()
  RETURNS TRIGGER AS $$
  BEGIN
    RAISE EXCEPTION
      'audit_logs is append-only. entry % attempted %', OLD.id, TG_OP
    USING ERRCODE = 'restrict_violation';
  END;
  $$ LANGUAGE plpgsql;
`);
await knex.raw(`
  CREATE TRIGGER audit_logs_no_update
  BEFORE UPDATE ON audit_logs
  FOR EACH ROW EXECUTE FUNCTION ams_forbid_audit_mutation();
`);
await knex.raw(`
  CREATE TRIGGER audit_logs_no_delete
  BEFORE DELETE ON audit_logs
  FOR EACH ROW EXECUTE FUNCTION ams_forbid_audit_mutation();
`);

```

backend/migrations/001_tenancy_identity_audit.js

Keyword in the trigger	What it means	Why it matters
BEFORE	Runs before the change is applied	The statement is aborted; the row is untouched
FOR EACH ROW	Once per affected row	A multi-row UPDATE is blocked row by row
RAISE EXCEPTION	Aborts the whole statement/transaction	The change never happens
ERRCODE = 'restrict_violation'	SQLSTATE 23000	The application receives a <i>recognisable</i> error class rather than a generic failure

Why BEFORE rather than AFTER

A BEFORE trigger can veto the change. An AFTER trigger runs once the row is already modified and can only *report* the problem — the write has happened. For a guarantee, the veto must come first. The consequence is that a blocked mutation rolls back entirely: no partial write, no partial audit entry.

```

-- Demonstrating the guarantee
UPDATE audit_logs SET actor_id = 'someone-else' WHERE id = 5001;
-- ERROR:  audit_logs is append-only. entry 5001 attempted UPDATE

DELETE FROM audit_logs WHERE id = 5001;
-- ERROR:  audit_logs is append-only. entry 5001 attempted DELETE

-- Not even a superuser avoids the trigger unless they disable it:
SET session_replication_role = replica; -- requires superuser
-- ... which is precisely the kind of act the external chain head detects.

```

The one remaining bypass, stated plainly

A superuser can disable triggers with `session_replication_role = replica`, or drop them. That is not a defect — it is the nature of a database administrator.

The design assumes such an act leaves another trace: the chain head is "additionally exported for external anchoring" per the module header, so a rewritten chain produces a different head than the anchored one.

No code implements the external anchoring. It is stated as a production intention. Without it, the protection reduces to "the database and its administrator are honest", which is a weaker claim than the header implies.

Hash chaining and what it does not prove

```
import { createHash } from 'node:crypto';

/** Genesis sentinel. Chaining starts here. */
export const GENESIS_HASH = '0'.repeat(64);

export function computeHash(entry, prevHash) {
  const canonical = canonicalise({
    id: entry.id,
    tenantId: entry.tenantId,
    actorId: entry.actorId,
    entityType: entry.entityType,
    entityId: String(entry.entityId),
    action: entry.action,
    before: entry.before,
    after: entry.after,
    ip: entry.ip,
    occurredAt: entry.occurredAt,
  });
  return createHash('sha256').update(prevHash, 'utf8').update(canonical, 'utf8').digest('hex');
}

export function chainEntry(entry, prevHash) {
  return computeHash(entry, prevHash || GENESIS_HASH);
}
```

```
// The chain in concrete form
Entry 1: hash(prev = 0000...0, content = "line 1 set to $10,000") → 8f2a...
Entry 2: hash(prev = 8f2a..., content = "line 1 set to $20,000") → 4c71...
Entry 3: hash(prev = 4c71..., content = "line 1 frozen") → b93d...

// Someone edits Entry 1 to say $50,000 and recomputes its own hash.
// Entry 2's stored prev_hash no longer matches. Verification fails at entry 2.
```

```

export function verifyChain(entries) {
  let prev = GENESIS_HASH;
  for (const entry of entries) {
    const expected = computeHash(entry, prev);
    if (expected !== entry.entryHash) {
      return {
        valid: false,
        entriesChecked: entries.indexOf(entry),
        brokenAt: entry.id,
        reason: 'Hash mismatch: entry content or ordering has been altered',
      };
    }
    prev = entry.entryHash;
  }
  return { valid: true, entriesChecked: entries.length, brokenAt: null, reason: null };
}

```

Property	PROVEN	NOT proven
Entry 1 altered	Chain breaks at entry 2	Who altered it, or with what credential
Entry 5 deleted	Chain breaks at entry 6	Whether the deletion was legitimate
Entries reordered	Chain breaks immediately	That reordering was accidental
Entire chain rewritten	Only by changing the head	Anything — if nothing external records the head
Entry written by the application	It is in the chain	That the underlying action was justified

The precise claim, and why precision matters here

The module says it "does not make the data trustworthy — it makes tampering DETECTABLE".

That is a real and valuable property. But a system that presents tamper-evidence as trustworthiness will eventually be asked to prove something it cannot prove. The correct statement to an auditor is: *any alteration of recorded history breaks the chain and is detectable on verification — not the history is correct.*

Canonicalisation and the phantom corruption bug

```

/**
 * Deterministic JSON serialisation.
 *
 * Object key order must be stable or the hash is meaningless – the same
 * logical entry would hash differently depending on insertion order, and
 * verification would report corruption that never happened.
 */

```

```

export function canonicalise(value) {
  if (value === null || typeof value !== 'object') return JSON.stringify(value ?? null);
  if (Array.isArray(value)) return `[$${value.map(canonicalise).join(',')}]`;
  const keys = Object.keys(** @type {Record<string, unknown>} */ (value)).sort();
  const parts = keys.map((k) => `${JSON.stringify(k)}:${canonicalise(** @type {any} */ (value)[k])}`);
  return `{${parts.join(',')}`;
}

```

```
canonicalise({ b: 2, a: 1 }) // '{"a":1,"b":2}'
canonicalise({ a: 1, b: 2 }) // '{"a":1,"b":2}' ← identical, by design

// Without the .sort(), these would hash differently and verification
// would report corruption on a chain that is perfectly intact.
```

A bug that would have been catastrophic and invisible

JavaScript preserves object key insertion order. Two objects with identical content but different insertion order serialise differently and therefore hash differently.

The failure mode is the worst kind: the system reports a problem that **does not exist**. Engineers would investigate a phantom corruption, and if they believed it they might "fix" it by disabling verification — removing the control because it produced false alarms.

This is a small amount of code (`.sort()`) preventing a large amount of wasted time and a potentially dangerous response.

```
export function verifyTenantChain(allEntries, tenantId) {
  const scoped = allEntries
    .filter((e) => e.tenantId === tenantId)
    .sort((a, b) => a.id - b.id);
  return { tenantId, ...verifyChain(scoped) };
}

export function chainHead(entries) {
  if (entries.length === 0) return GENESIS_HASH;
  const last = [...entries].sort((a, b) => a.id - b.id).at(-1);
  return last.entryHash;
}
```

Per-tenant verification, and why it is necessary

A single global chain means one tenant's corruption would be reported against a shared sequence, contaminating every other tenant's audit result. For a multi-tenant SaaS, that is unacceptable: customer A's bad row would make customer B's audit fail.

`verifyTenantChain` scopes first, then verifies. Each customer's audit is provable independently, which is also what an auditor examining one customer's records would require.

```

export function auditAnomalies(entries) {
  const findings = [];
  const ordered = [...entries].sort((a, b) => a.id - b.id);

  ordered.forEach((e, i) => {
    if (i > 0 && e.id <= ordered[i - 1].id) {
      findings.push({ severity: 'SEV1', code: 'OUT_OF_ORDER', entryId: e.id, detail: 'Non-monotonic entry id' });
    }
    if (!e.actorId || e.actorId === 'system') {
      findings.push({ severity: 'INFO', code: 'SYSTEM_ACTOR', entryId: e.id, detail: 'Entry written by system actor' });
    }
    if (e.action === 'DELETE' && e.before === null) {
      findings.push({ severity: 'SEV2', code: 'DELETE_WITHOUT_SNAPSHOT', entryId: e.id, detail: 'Delete recorded with no record of what was deleted' });
    }
    if (e.action === 'APPROVE' && e.entityType === 'cost_authority'
      && String(e.actorId) === String(** @type {any} */ (e.before)?.approved_by ?? '')) {
      findings.push({ severity: 'SEV1', code: 'SELF_APPROVAL', entryId: e.id, detail: 'Approver matched the authoriser of the request' });
    }
  });

  return {
    findings,
    highestSeverity: findings.reduce(
      (worst, f) => (worst === null || rank(f.severity) < rank(worst) ? f.severity : worst),
      /** @type {string|null} */ (null),
    ),
  };
}

```

Hash verification answers "was this changed?". Anomaly detection answers "was this a legitimate thing to do?". Both are needed, and they catch different problems.

Finding	Severity	Meaning
OUT_OF_ORDER	SEV1	Entries reordered or rewritten
SELF_APPROVAL	SEV1	The same person authorised what they requested — a segregation-of-duties breach
DELETE_WITHOUT_SNAPSHOT	SEV2	A deletion recorded with no record of what was deleted
SYSTEM_ACTOR	INFO	Written by an automated process, not a person

```

export const SOD_RULES = [
  'SELF_APPROVAL',           // approver === requester
  'EVALUATOR_IS_AUTHOR',     // evaluator also authored the sourcing event
  'PAYMENT_RELEASER_IS_ORIGINATOR',
  'SUPPLIER_BIDS_OWN_TENDER',
  'TECHNICAL_APPROVES_OWN_CHECK',
  'PERIOD_CERTIFIER_WITHOUT_AUTHORITY',
];

```

shared/src/domain.js — the rules the anomaly detector implements. Note that only the first is currently detected; the other five are declared but not enforced anywhere.

Five of six segregation-of-duties rules are declared but not enforced

SOD_RULES lists six violations. `auditAnomalies` detects one (SELF_APPROVAL), and only for `cost_authority` entities. The other five have no implementation.

This is the honest reading: the vocabulary is defined, the detection for one rule exists, and the remaining five are a specification rather than a feature. `EVALUATOR_IS_AUTHOR` in particular — an evaluator who also wrote the tender — is a procurement fraud pattern that most control frameworks treat as mandatory.

```
CREATE TABLE audit_logs (  
  id          BIGSERIAL PRIMARY KEY,  
  tenant_id   TEXT NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,  
  actor_id    TEXT,  
  actor_ip    INET,  
  entity_type TEXT NOT NULL,  
  entity_id   TEXT NOT NULL,  
  action      TEXT NOT NULL,  
  before_json JSONB,  
  after_json  JSONB,  
  correlation_id UUID,  
  occurred_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
  prev_hash   CHAR(64) NOT NULL,  
  entry_hash  CHAR(64) NOT NULL  
)  
`);  
await knex.raw(`CREATE INDEX idx_audit_tenant ON audit_logs (tenant_id, id)`);  
await knex.raw(`CREATE INDEX idx_audit_entity ON audit_logs (entity_type, entity_id)`);
```

The `idx_audit_tenant` index on `(tenant_id, id)` matches how `verifyTenantChain` reads: filter by tenant, order by id. The composite index serves both the filter and the sort from one structure.

Hash chaining has a concurrency hole that is not addressed

Computing entry N+1's hash requires reading entry N's hash. If two audit entries for the same tenant are written concurrently, both read the same `prev_hash` and both chain from it — producing two entries with the same predecessor. One of them breaks the chain.

The fix is the same pattern as the budget line: `SELECT ... FOR UPDATE` on the tenant's chain head before inserting, so writes to one tenant's chain serialise. **No such lock exists in the codebase.** The chain head lock would need to be taken in the same transaction as the business change, which is exactly the structure described in Part I.

This is a genuine gap. It is not currently exploitable because no server writes audit entries, but it must be addressed before one does.

Part VI — Side effects that cannot be lost or duplicated

A committed financial change frequently needs a side effect: send an email, produce a bank file, notify a supplier, update a carbon report. Getting that correct is a genuinely hard problem, and the schema in migration 001 already anticipates it.

```
// ----- transactional outbox -----
// Every side effect (email, SMS, bank file, carbon report) is written
// here in the same transaction as the business change, then delivered by
// a worker. This is why a committed change can never lose its side effect.
await knex.raw(`
  CREATE TABLE outbox (
    id                BIGSERIAL PRIMARY KEY,
    tenant_id         TEXT NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    aggregate_type     TEXT NOT NULL,
    aggregate_id       TEXT NOT NULL,
    message_type       TEXT NOT NULL,
    payload            JSONB NOT NULL,
    created_at         TIMESTAMPTZ NOT NULL DEFAULT now(),
    published_at       TIMESTAMPTZ,
    attempts           SMALLINT NOT NULL DEFAULT 0,
    last_error         TEXT
  )
`);
await knex.raw(`
  CREATE INDEX idx_outbox_unpublished ON outbox (created_at)
  WHERE published_at IS NULL
`);
```

backend/migrations/001_tenancy_identity_audit.js

The problem: a commit that must send an email

Consider approving a purchase commitment. The approval must be recorded, money must move, and the requester must be notified. These are two systems: a PostgreSQL database and an email provider. There is no transaction spanning both.

```
// The naive approach
await db.transaction(async (trx) => {
  await trx('commitments').insert({ /* ... */ });
  await trx('budget_lines').increment('committed_cents', amountCents);
});

await sendEmail({ to: requester, subject: 'Approved', body: '...' });
```

Failure	Result
Process crashes between commit and send	Money committed, no notification. The requester does not know.
Email provider is down	Money committed, no notification , and an exception thrown after the commit — which might be caught and retried, causing a double approval.
Email sends, response is lost	Two notifications for one approval, or a retry that commits again.
Database commit fails	An email announces an approval that did not happen.

The dual-write problem

This is called the *dual-write problem*, and it has no correct solution when you attempt to coordinate two systems that cannot share a transaction. Every mitigation trades one failure mode for another.

The standard fix is to stop treating the email as a separate act. Record the *intent* to send, atomically with the business change, and let a separate process turn the intent into an email.

The transactional outbox

```
// Step 1 – one transaction: the business change AND the intent to notify
await db.transaction(async (trx) => {
  await trx('commitments').insert({ /* ... */ });
  await trx('budget_lines').increment('committed_cents', amountCents);

  // The email is now recorded. If we crash here, nothing happened at all.
  await trx('outbox').insert({
    tenant_id: tenantId,
    aggregate_type: 'cost_authority',
    aggregate_id: commitmentId,
    message_type: 'COMMITMENT_APPROVED',
    payload: JSON.stringify({ amountCents, requesterEmail }),
  });
});

// Step 2 – a separate worker, possibly minutes later
const pending = await db('outbox')
  .whereNull('published_at')
  .orderBy('created_at')
  .limit(100);

for (const message of pending) {
  try {
    await sendEmail(JSON.parse(message.payload));
    await db('outbox').where({ id: message.id })
      .update({ published_at: db.fn.now() });
  } catch (err) {
    await db('outbox').where({ id: message.id }).increment('attempts', 1);
  }
}
```

What the outbox eliminates

The "lost side effect" failure is now impossible. If the transaction commits, the notification is recorded. If it rolls back, neither the commitment nor the notification exists. There is no window in which money moved and the notification did not.

This is the same principle as FOR UPDATE and the CHECK constraints: move the invariant into a place that cannot be bypassed. Here it is "the intent to notify is atomic with the change" — enforced by the database transaction itself.

Failure	Naive approach	With outbox
Crash after commit, before send	Notification lost forever	Impossible. The intent was committed.
Email provider down	Commit succeeds, exception thrown	Recorded, retried by the worker.

Failure	Naive approach	With outbox
Worker crashes mid-batch	n/a	Un-published rows remain; the next worker picks them up.
Notification delivered twice	Possible on retry	Possible — see below.

What the outbox does NOT give you, and the honest statement

Delivery is **at-least-once**, not exactly-once. If the worker sends the email successfully and then crashes before writing `published_at`, the next run sends it again.

This is a real and unsolved problem: **exactly-once delivery across a network boundary does not exist**. The mitigations are all partial:

- Make the recipient idempotent (a visible reference number, so a duplicate is recognisable) - Use an email provider that supports idempotency keys
- Accept duplicates for messages where they are harmless, and never for messages that trigger money movement

The schema anticipates retries with `attempts` and `last_error`, and the partial index serves the worker's query.

No worker exists and no delivery policy is documented. This should be settled before any notification is wired up.

```
await knex.raw(`
  CREATE INDEX idx_outbox_unpublished ON outbox (created_at)
  WHERE published_at IS NULL
`);
```

Why the index is partial

A partial index on `WHERE published_at IS NULL` stores only unpublished rows. Once a message is published its index entry disappears.

For a table holding every notification ever sent, this is the difference between an index that grows without bound and one that stays proportional to the backlog. The worker's query — "what is unpublished, oldest first" — is exactly the index's predicate and ordering.

This is the correct pattern, and it also appears in the commitments table: `idx_commitments_open ... WHERE movement_type IN ('PIPELINE', 'COMMITMENT')`.

The outbox has no claim mechanism, and a naive worker will duplicate

The loop above reads pending messages and sends them. If two workers run concurrently — which is the normal case for a reliable queue — both read the same rows and both send.

The correct pattern is `FOR UPDATE SKIP LOCKED`:

```
-- Correct worker claim: each row goes to exactly one worker
SELECT id, payload FROM outbox
  WHERE published_at IS NULL
 ORDER BY created_at
FOR UPDATE SKIP LOCKED      -- skip rows another worker already holds
LIMIT 100;

-- Process them, then set published_at. The row locks prevent overlap.
```

`SKIP LOCKED` is the crucial keyword: instead of waiting for a locked row, the worker steps over it to the next available one. That is what allows horizontal scaling without coordinating the workers at all. **This is not**

implemented. No worker exists. It is recorded here because the schema is designed for it and the eventual implementation must use it.

What the outbox guarantees and what it does not

Property	Guaranteed?	Mechanism
The business change and the intent to notify are atomic	Yes	One database transaction
A committed change always has a recorded notification	Yes	Same transaction
A rolled-back change has no notification	Yes	Same transaction
Notifications are eventually attempted	Yes , given a running worker	Partial index polling
A notification is delivered exactly once	No	At-least-once delivery
A notification is delivered in order	No	Parallel workers; no sequence guarantee
The notification content is current at send time	No	The payload is a snapshot from commit time

Snapshot payloads and stale notifications

The payload is serialised into the row at commit time. If the commitment is subsequently reversed, the pending notification still says "approved" and will be sent with no correction.

Two defensible designs exist: send the snapshot and accept that a rarely-concurrent reversal produces a stale message, or have the worker re-read current state before sending. The first is simpler and faster; the second is accurate. **No decision is recorded**, and it should be before notifications exist, because changing it later is expensive.

Part VII — Verification as an engineering discipline

AMS has 190 tests. That number is not the point. The point is what those tests can and cannot find, and this part is explicit about the boundary.

Four kinds of test and which defects each finds

Kind	What it asserts	Finds	Blind to
Expected value	A hand-written answer	Regressions in known cases	Anything the author did not think of — including errors in their own expectation
Structural / property	An invariant that must always hold	Whole classes of defect	Cases where the invariant is the wrong invariant
Differential	Two implementations agree	Bugs where one implementation is wrong	Bugs where both are wrong the same way
Metamorphic	A known transformation preserves a relation	Bugs without needing a reference	Bugs in the relation itself

```
describe('defect 1 – no fabricated metrics reach the API', () => {
  it('the aggregate carries no internal or underscore-prefixed fields', () => {
    const agg = aggregatePnl([LEG(150), LEG(160)]);
    const offenders = Object.keys(agg).filter((k) => k.startsWith('_'));
    expect(offenders).toEqual([]);
  });

  it('every aggregate metric is a real sum of real flight facts', () => {
    const flights = [LEG(150), LEG(160), LEG(140)];
    const agg = aggregatePnl(flights);
    // ...
    expect(agg.passengers).toBe(450);
  });
});
```

tests/costing-defects.test.js — a structural test. It would fail regardless of what the correct numbers are.

```
/**
 * Regression tests for two defects found by REVIEW rather than by any
 * failing test. Both shipped green through 118 tests.
 *
 * DEFECT 1 – aggregatePnl computed a `roll` object from fabricated input
 * (distanceKm: 0, passengers: 0, a `seatsFrom` helper that multiplied by
 * an undefined field and therefore always returned 0). It produced an
 * object of near-zero metrics and attached it to the return value as
 * `_roll`, so it serialised into every route P&L API response.
 *
 * DEFECT 2 – breakEvenPassengers computed
 * (revenue / X) * 0      <- multiplied by ZERO
 * and fell back to a hardcoded US$120 fare x 0.62.
 * The number it returned was derived from nothing but a magic constant.
 *
 * The lesson these encode: a test suite that only asserts what the code
 * intends to do will never catch a function whose intent was never
 * implemented. These tests assert STRUCTURAL properties – no fabricated
 * fields reach the API, and no magic constant influences an answer.
 */
```

```
// DEFECT 2 – before and after
function breakEvenPassengers(agg) {
  return (agg.revenueCents / agg.loadFactorPpm) * 0; // always 0
  const HARDCODED_FARE = 12_000;
  return HARDCODED_FARE * 0.62; // always 7440
}

function breakEvenPassengers(agg) {
  const flights = agg.flights || 1;
  const passengers = agg.passengers || 0;

  const revenuePerPassengerCents = passengers > 0
    ? Math.round(agg.revenueCents / passengers) : 0;
  const marginalCostCents = agg.marginalSeatCostCents ?? 0;
  const contributionPerPassengerCents = revenuePerPassengerCents - marginalCostCents;

  const perFlightLoss = Math.max(0, -agg.contributionCents) / flights;
  if (perFlightLoss === 0) {
    return { additionalPassengersPerFlight: 0, closeableByLoadFactor: true,
      revenuePerPassengerCents, contributionPerPassengerCents };
  }

  // Carrying a passenger costs more than they pay. More load does not
  // fix this; the route needs re-pricing, a fleet change or withdrawal.
  if (contributionPerPassengerCents <= 0) {
    return { additionalPassengersPerFlight: Number.MAX_SAFE_INTEGER,
      closeableByLoadFactor: false,
      revenuePerPassengerCents, contributionPerPassengerCents };
  }

  return {
    additionalPassengersPerFlight: Math.ceil(perFlightLoss / contributionPerPassengerCents),
    closeableByLoadFactor: true,
    revenuePerPassengerCents,
    contributionPerPassengerCents,
  };
}
```

What the fix reveals that the original could not express

The original function had no way to represent the case where a passenger costs more to carry than they pay.

There was no field for it, so the branch could not exist.

In that situation more load makes the loss **worse**, and the honest answer is that load factor cannot fix the problem.

The corrected version returns `closeableByLoadFactor: false`.

A function that can only ever produce a positive answer is usually hiding a case. When you review a calculation, ask what business situations it cannot represent.

Situation	Correct answer	What the old code said
Loss, passengers profitable	A real number of extra passengers	7440, always
Loss, each passenger costs more than they pay	Cannot be closed by load factor	7440
Already profitable	0	7440

Oracle design and seeding

```

/**
 * Verification suite for two optimisation algorithms.
 *
 * These tests do not assert what the code intends to do. They assert
 * agreement between three INDEPENDENT implementations, plus structural
 * properties that must hold for any correct answer.
 *
 * 1. solveRoute      - Pareto label-setting (production)
 * 2. solveRouteDP    - fuel-indexed DP (reference)
 * 3. bruteForceRoute - exhaustive enumeration (oracle)
 *
 * If all three agree on thousands of random instances, and the oracle is
 * correct by construction, the production algorithm is correct on those
 * instances. That is meaningfully stronger than a test suite written
 * from the same mental model as the implementation – which is how the
 * `_roll` defect shipped through 118 green tests.
 */

```

```

function randomInstance(rand, n) {
  const nodes = Array.from({ length: n }, (_, i) => `N${i}`);
  const flights = [];
  for (const from of nodes) {
    for (const to of nodes) {
      if (from === to) continue;
      if (rand() < 0.42) continue; // sparse
      flights.push({
        from, to,
        costCents: int(rand, 1, 5000),
        fuelKg: int(rand, 1, 12),
      });
    }
  }
  return {
    flights,
    source: nodes[0],
    destination: nodes[n - 1],
    tankCapacityKg: int(rand, 10, 40),
    minReserveKg: int(rand, 0, 6),
  };
}

```

Two properties of this generator matter more than the rest of the function. **Cost and fuel are drawn independently**, so genuinely incomparable cost/fuel trade-offs arise — which is what exercises the dominance rule. And **the density is ~58%**, sparse enough to produce disconnected networks and dead ends, dense enough to produce many candidate routes.

```

function mulberry32(seed) {
  let a = seed >>> 0;
  return function rand() {
    a |= 0; a = (a + 0x6d2b79f5) | 0;
    let t = Math.imul(a ^ (a >>> 15), 1 | a);
    t = (t + Math.imul(t ^ (t >>> 7), 61 | t)) ^ t;
    return ((t ^ (t >>> 14)) >>> 0) / 4294967296;
  };
}

```

The seed is derived from the loop counter, so test n always produces the same instance. A failure at seed 3 is a permanent, shareable regression case rather than a rumour.

Guarding against vacuous tests

```

    compared += 1;
    if (oracle.feasible) feasible += 1;
    expect(pareto.feasible, `seed ${seed}: feasibility must match oracle`).toBe(oracle.feasible);
    // ...
  }
  expect(compared).toBeGreaterThan(2500);
  // The test is worthless if every instance was trivially infeasible.
  expect(feasible / compared).toBeGreaterThan(0.3);

```

The most underrated line in the suite

If a change to the generator produced only infeasible networks, every comparison would trivially pass — both solvers would correctly report "no route" — and the test would prove nothing while reporting green.

A test suite that cannot fail is worse than no test suite, because it manufactures confidence. Asserting that your test data is interesting is a necessary and frequently omitted step.

The suite goes further and includes a test whose job is to prove the *other* tests are meaningful:

```

describe('the fuel constraint actually binds', () => {
  it('finds many instances where the unconstrained shortest path is INFEASIBLE', () => {
    // ...
    // A naive Dijkstra would be wrong this often. Proof the constraint matters.
    expect(disagreements).toBeGreaterThan(50);
  });
});

```

The defect ledger

#	Defect	Type	Found by	Would a test have found it?
1	Fabricated <code>_roll</code> object in <code>aggregatePnl</code>	Wrong data returned	Reading the code	No — 118 green tests
2	<code>breakEvenPassengers</code> multiplied by zero	Wrong arithmetic	Reading the code	No — 118 green tests
3	<code>requiredAtDeparture</code> under-fuelled first departure	Wrong formula	Exhaustive oracle	No — the test agreed with it
4	DP carried departure node instead of arrival node	Wrong state	Cross-implementation check	No — no existing test
5	Feasibility check demanded whole route fit in tanks	Wrong condition	Exhaustive oracle	No
6	Infinite loop on identical labels	Non-termination	Test suite timeout	No — not a wrong answer

```
// One defect, in full – the infinite loop

// A zero-cost, zero-fuel cycle produces a label identical to one already
// present. Identical labels do not strictly dominate one another, so
// without this the planner re-queues the same (cost, fuel) pair forever
// and never terminates.

    for (const other of target) {
        if (dominates(other.costCents, other.fuelUsedKg, costCents, fuelUsedKg)) {
            dominated = true; break;
        }
        if (other.costCents === costCents && other.fuelUsedKg === fuelUsedKg) {
            dominated = true; break;
        }
    }
}
```

Pattern	Count	Implication
Found by reading code	2	The test suite was not looking for these
Found by differential testing	3	The highest-yield technique in the codebase
Found by the test simply running	1	Non-termination is invisible to assertions
Code and test wrong together	1	The normal failure mode of hand-written expectations

Where software verification ends

Area	What the code guarantees	What requires a qualified human
Cost taxonomy	Categories are valid; totals reconcile; parts sum exactly	Whether the categories match IFRS and IATA reporting requirements
Regulatory reserve	Fuel never drops below the configured figure	Whether the configured figure is legally correct for the jurisdiction
MEL intervals	Deadlines computed from the category table, UTC, end-of-day	Whether the category table matches current EASA CS-GEN-MMEL
AD applicability	Serial range and threshold rules applied as given	Whether the model matches how a regulator actually determines applicability
Emission factors	Tonnes computed from the configured factor	Whether the factor is correct; whether the operator's own approved MRV method is required instead
Fuel burn coefficients	Marginal seat cost computed from the constants	Validation against this fleet's measured data
Operating policy	The numbers are computed and shown	What the airline should do about them

The boundary, stated as precisely as it can be

Every test in AMS proves the code does what its authors intended. **No test can prove the intent was correct.**

` // What the system proves: // "Cumulative fuel on this route never exceeds capacity minus the // configured reserve of 1500 kg." // // What it cannot prove: // "1500 kg is the legally correct reserve for this operation." `

If the regulatory reserve requirement is wrong, all 190 tests pass and the aircraft is dispatched below its legal minimum. Software can enforce a rule; only a qualified person can establish that the rule is the right one.

This is not a limitation to be engineered away. It is a permanent division of responsibility, and pretending otherwise is how systems cause harm.

Part VIII — Engineering judgement and review

The most useful thing this document can give you is a way to read the codebase for risk — and a question list you can apply to any money-touching or safety-touching function without reading the whole file.

Reading the codebase for risk

The modules are not equally risky. Knowing where the risk is concentrated determines where a reviewer should spend their hour.

Module	What a wrong answer causes	Verification	Risk
money.js	Every figure in the system drifts	Directly tested; exactness is structural	Low
routing/constrained-path.js	An unflyable route is dispatched	Three implementations cross-checked; 190 tests	Low
metrics.js	Commercial ratios are wrong	Tested; recomputation discipline documented	Low
audit/chain.js	Tampering becomes undetectable	Simple, self-contained; soundness argued	Low
domain.js	Nothing — it is data	Used as the source for DB constraints	Very low
costing.js	Route decisions are wrong	Tested, but two defects shipped green	Medium
carbon.js	A compliance misstatement; a double-counted obligation	Partially tested; one rounding defect found	Medium
airworthiness.js	An aircraft flies with an expired deferral	Tested against researched rules; not verified by a licensed professional	HIGH
fuel.js	Tankering decisions are mispriced	Tested; constants are industry assumptions	Medium
cost-authority.js	The airline overspends	Concurrency fixed by lock; one live defect	HIGH

The two HIGH-risk modules are the two where a wrong answer is unrecoverable: money that leaves the bank account, and an aircraft that is not legally airworthy. Neither is the most complex code in the system, and neither is where a naive reviewer would look first.

The questions to ask of any money-touching function

```
// Read the function. Then ask, in this order:

// 1. UNITS
//   What unit is every number in? Cents, kg, ppm, hours?
//   Is the unit in the name? (amountCents not amount)
//   If two numbers are multiplied, is the resulting unit what the caller expects?

// 2. VALIDATION
//   Does every monetary input pass through assertSafeInteger or intOr?
//   Is a float rounded, or does it throw?
//   Throwing is correct. Rounding a float that arrived from upstream hides a bug.

// 3. ROUNDING
//   Where exactly is the rounding, and is it stated in a comment?
//   Is it rounded once at creation, or repeatedly downstream?
//   Is the rule symmetric about zero? (Math.round(-0.5) === -0)

// 4. RECONCILIATION
//   Do the parts provably sum to the whole?
//   Is there an assertReconciles or equivalent self-check?
//   Is there a database constraint that enforces it independently?

// 5. ZERO AND NEGATIVE
//   What happens at zero? At negative? At null?
//   Is `??` used where zero carries meaning, rather than `||`?

// 6. PRECISION
//   Could this value be large enough that the chosen unit rounds to zero?
//   CASK in cents rounds to 0 for a long haul. Microcents exist for this.
```

```
// Applied to the DEFECT 2 function – what the questions reveal

// UNITS:      revenueCents ✓, loadFactorPpm ✓ – names are good
// VALIDATION: no check that the division is meaningful
// ROUNDING:   the bug – `(revenue / X) * 0` rounds nothing, it erases everything
// RECONCILIATION: none applicable
// ZERO:       no field could represent "load factor cannot fix this"
// PRECISION:  not applicable

// Question 6 and the "what business case can't you express" question
// together would have surfaced that the function had no answer for
// an unprofitable passenger.
```

The questions to ask of any database schema

```
// 1. TYPES
// Is every money column BIGINT (or NUMERIC)? Any FLOAT is a defect.
// Is the unit visible in the column name? (budgeted_cents, not amount)
// Are non-integer quantities stored as NUMERIC rather than forced to int?
// aircraft.current_hours NUMERIC(14,2) – correct. Flight hours are fractional.

// 2. CONSTRAINTS
// Is every enumerated column constrained to its allowed values?
// Is there a CHECK for every invariant the application relies on?
// Would the system be safe if a new client ignored every application rule?

// 3. TENANCY
// Does every table have tenant_id NOT NULL?
// Is RLS enabled? Is it FORCED? (ENABLE alone protects nothing if the
// application connects as the table owner – the default)
// Is the tenant set scoped to the transaction, so pooling cannot leak it?

// 4. DELETION
// What happens when a tenant is deleted? CASCADE destroys audit evidence.
// Is there a soft-delete or CLOSED status path? (tenants.status has one;
// nothing implements the path)

// 5. DERIVED VALUES
// Is anything computed stored rather than generated?
// Any stored derived value is a drift risk. GENERATED ALWAYS cannot drift.

// 6. REFERENCES
// Do foreign keys CASCADE or RESTRICT? Each choice is a decision.
// Is ON DELETE CASCADE right for a parent that a regulator may ask about?
```

```
// Applied to migration 002 – a real finding

// Q1: budget_lines uses BIGINT for money ✓, with _cents in every name ✓
// Q2: every state column CHECK-constrained ✓
// available_cents >= 0 ✓
// Q3: every table has tenant_id NOT NULL ✓, RLS ENABLE + FORCE ✓
// But: app.tenant_id is a connection setting. Pooling is the hazard. Not handled.
// Q4: aircraft, budget_lines etc. CASCADE from tenants. Destroys audit evidence.
// tenants.status includes 'CLOSED' – the safe path exists in the schema
// and is not implemented anywhere.
// Q5: available_cents is GENERATED ALWAYS ✓ – the best pattern in the schema.
// But availabilityCents() duplicates the formula in JavaScript with no
// automated check that they agree.
// Q6: budget_lines → budgets CASCADE. Deleting a draft budget removes its lines.
// Defensible, but worth a deliberate decision.
```

```
// 7. CONCURRENCY
// Is any read-modify-write in the application layer?
// Read in JS, add, write the result → lost update.
// Use .increment() / a single UPDATE ... SET x = x + $1.
// Is any check-then-write outside a lock?
// → TOCTOU. See Part I.
// In a multi-row transaction, is the lock order deterministic?
// → otherwise deadlock is possible.

// 8. ATOMICITY
// Does a business change and its audit entry land in one transaction?
// Does a business change and its notification intent land in one transaction?
// Is there an outbox? (migration 001 has one ✓)

// 9. IMMUTABILITY
// Should any table be append-only? Is that enforced by trigger, or only convention?
// (audit_logs has triggers ✓ – the mechanism, not the policy)
```

Known gaps, stated plainly

These are recorded so they are not discovered later. Nothing here is a suggestion; each is a known limitation of the current code.

Gap	Impact	Severity
<code>commitmentId</code> && <code>commitment.budget_line_id</code> in <code>settleCommitment</code> (<code>cost-authority.js:361</code>)	An accidental logical AND where an assignment was intended. Currently evaluates correctly; would silently fail if <code>commitmentId</code> were ever an empty string.	Fix before production
Audit chain has no head lock	Concurrent audit writes for one tenant can produce two entries with the same predecessor, breaking the chain.	Fix before audit writes exist
Outbox has no SKIP LOCKED claim	Two concurrent workers would deliver the same message twice.	Fix before a worker exists
No external chain-head anchoring	The audit trail detects partial tampering only. A full rewrite is undetectable.	Design decision needed
<code>strict: false</code> in <code>tsconfig</code>	Null and undefined are not rigorously checked in code that handles money and airworthiness.	Should be <code>true</code>
No CI pipeline	<code>npm run typecheck</code> must be run by hand. Nothing enforces it.	Process gap
Availability formula duplicated	<code>availabilityCents()</code> in JS and the generated column in SQL must agree. No test asserts it.	Add a regression test
Tenant deletion CASCADE	Deleting a tenant destroys audit evidence irreversibly.	Needs a compliance decision
Five of six SOD rules unenforced	Declared in <code>SOD_RULES</code> , detected for one rule only.	Scope decision
No server, no authentication, no API	The system cannot be run. No request can be authenticated; no tenant context is set.	Blocking
Server identity Knex typedef is a stand-in	The cost-authority module has never been executed against a real database.	Blocking
Regulatory constants unverified	MEL intervals, emission factors, burn coefficients are researched but not validated by a qualified professional.	Blocking for regulated use

```
// The stand-in that blocks execution of the control core:

/**
 * Structural stand-in for the Knex instance, declared locally so the
 * type-check gate works without the runtime dependency installed. When
 * `knex` lands, replace this with `import('knex')` and delete it.
 *
 * @typedef {Object} Knex
 * @param {string} tableName
 * @returns {any} QueryBuilder
 */
```

What "blocking" means here, precisely

The pure domain modules — money, metrics, costing, airworthiness, fuel, carbon, routing — are tested and can be relied on as calculations.

The modules that touch a database — cost-authority, audit chain — have **never been executed**. Their SQL is written but unrun, their type is a stand-in, and the concurrency behaviour described in Part I is documented from PostgreSQL's specification rather than observed.

Everything in this document about concurrency, row-level security and the outbox is engineering reasoning about code that has not run. That is not a criticism of the reasoning — it is the accurate scope of what is known.

What would make this system genuinely production-ready

1. **Write the server, with authentication and tenant context set per transaction.** Everything in Parts I, III and VI is blocked on this. Use `FOR UPDATE SKIP LOCKED` for any worker.
2. **Execute the migrations against a real PostgreSQL 16 instance and write integration tests that actually hit the database.** The concurrency claims must be *observed*, not inferred.
3. **Add a concurrency test that would fail without the lock.** Spawn parallel authorisation requests against one budget line and assert that the committed total never exceeds availability. This is the test the codebase most conspicuously lacks.
4. **Fix the three defects identified in this document** and add regression tests for each.
5. **Turn on** `strict: true` and work through what it finds.
6. **Wire up CI** so `npm test` and `npm run typecheck` gate every change.
7. **Add the missing regression test** asserting `availabilityCents()` and the generated column agree.
8. **Obtain qualified sign-off** on the regulatory constants: MEL intervals, emission factors, the reserve model, AD applicability logic, the cost taxonomy under IFRS and IATA. No amount of testing substitutes for this.
9. **Instrument the routing solver** — log `maxFrontier` in production to turn the $O(L)$ assumption into a measurement.
10. **Decide the deletion and notification policies** and implement them before they are needed.

The single most valuable next step

Not any of the above individually — the **integration test that reproduces the concurrency scenario**. It is the one class of defect this codebase has no protection against, it is the one that costs real money, and it is the one that no amount of unit testing will find.

Everything else can be added incrementally. That one test either confirms the central control works or reveals that it does not.

Appendix — Reference tables

Concurrency failure modes

Failure	Shape	Detection	Mechanism that prevents it
Lost update	Read, modify, write. Two writers overwrite each other.	Only with concurrent writers	UPDATE $x = x + n$, or a row lock
TOCTOU / double spend	Check, gap, act on a stale value	Only under load	SELECT . . . FOR UPDATE inside the same transaction
Deadlock	Two transactions each hold what the other needs	PostgreSQL detects after <code>deadlock_timeout</code> (1s)	A deterministic global lock order
Dirty read	Reading uncommitted data from another transaction	Immediately visible	READ COMMITTED or above (the default)
Non-repeatable read	Re-reading a row and getting a different committed value	Requires a long transaction	REPEATABLE READ or SERIALIZABLE
Phantom read	A row matching a predicate appears mid-transaction	Requires a re-query	SERIALIZABLE, or a predicate lock
Duplicate side effect	A side effect performed twice after a retry	Requires a lost response	An idempotency key plus a unique index
Chain fork	Two concurrent writers read the same predecessor	Only detected at verification time	A lock on the chain head (not implemented)

Integer scale in PostgreSQL

Type	Bytes	Min	Max	Money range (as cents)
SMALLINT	2	−32,768	32,767	about \$327
INTEGER	4	−2.1 billion	2.1 billion	about \$21 million
BIGINT	8	−9.2 quintillion	9.2 quintillion	about \$92 quadrillion
JavaScript safe integer	8	−2 ⁵³ +1	2 ⁵³ −1	about \$90 trillion

Boundary	Value	Comment
SERIAL	2,147,483,647	PostgreSQL runs out of ids well before the range ends
BIGSERIAL	9.2 quintillion	Used for <code>audit_logs.id</code> and <code>outbox.id</code>
PostgreSQL index row limit	about 8,000 pages	An index over tens of millions of rows needs care

Milliseconds since the epoch

```
// AMS stores dates as milliseconds since the Unix epoch (1970-01-01T00:00:00Z)

const DAY_MS = 86_400_000;

// MEL deadlines – all UTC, which is not optional
export function rectifyDeadline(category, discoveredAtMs) {
  const spec = MEL_CATEGORIES[category];
  if (!spec) throw new TypeError(`Unknown MEL category "${category}"`);
  if (spec.days === null) return null; // Category A: no standard interval

  const discovery = new Date(discoveredAtMs);
  const due = new Date(discovery.getTime());
  // The day of discovery is excluded, so counting starts at day 1 = the
  // following calendar day.
  due.setUTCDate(due.getUTCDate() + spec.days);
  due.setUTCHours(23, 59, 59, 999); // valid to the END of that day
  return due.getTime();
}
```

Every method is a UTC method. The code uses `setUTCDate` and `setUTCHours`, never `setDate` and `setHours`. That is deliberate: a legal deadline must not shift by an hour when a jurisdiction changes its clocks, and it must not depend on the server's local timezone setting.

The off-by-one this function exists to get right

Categories B, C and D are "N calendar days **excluding the day of discovery**", running to the **end** of the resulting day.

A Category B defect found at 09:00 on Monday is due at 23:59 on Thursday. Naive duration arithmetic gives Thursday 09:00 — which is an hour early, meaning an aircraft could be dispatched on a Thursday morning flight with an expired deferral.

The `setUTCHours(23, 59, 59, 999)` is what makes it the end of the day rather than the same time of day.

Glossary

ACID

Atomicity, Consistency, Isolation, Durability — the four properties of a database transaction.

Advisory lock

A lock a transaction takes voluntarily, which the database does not enforce. Distinct from a row lock, which it does.

At-least-once delivery

A message may be delivered one or more times, never zero times. The strongest guarantee available across a network.

BIRD algorithm

Bottleneck-Induced Resource Dominance — the family of label-setting algorithms for resource-constrained shortest path. AMS uses a variant of it.

Canonicalisation

Converting a value to a single deterministic text form so that identical content always produces identical output. Required for hashing.

CHECK constraint

A database rule that rejects any row violating it. Applies to every write by every client.

Commit

The point at which a transaction's changes become permanent and visible. Nothing before it can be relied on.

Deadlock

Two or more transactions each holding a resource the others need, so none can proceed.

Differential testing

Testing by comparing two implementations against each other, rather than against expected values.

Dual-write problem

The difficulty of keeping two systems without a shared transaction consistent. Solved by an outbox.

Exclusive lock

A lock that prevents any other transaction from reading or writing the locked row. What `FOR UPDATE` takes.

FOR UPDATE

A PostgreSQL clause that locks selected rows until the current transaction ends. The mechanism that prevents double spending in AMS.

Generated column

A column whose value the database computes from an expression. `GENERATED ALWAYS ... STORED` cannot be written to directly and cannot drift from its inputs.

Graceful degradation

A system that loses a capability rather than failing entirely when a dependency is unavailable.

Idempotency key

A unique value sent with a request so that a retry does not perform the work twice.

ISOLATION LEVEL

The rule determining what a transaction can see of concurrent work. PostgreSQL defaults to `READ COMMITTED`, which permits the TOCTOU race.

Livelock

Transactions that keep changing state without progressing, though nothing is blocked. Distinct from deadlock.

Lost update

A write silently discarded because a concurrent transaction wrote a stale value.

Metamorphic test

A test asserting that a known transformation of the input produces the expected transformation of the output.

Monotonicity

The property that increasing a constraint never improves the answer. Routing monotonicity tests assert exactly this.

Oracle

In testing, an implementation correct by construction — usually slow and simple — used to validate a fast implementation.

Outbox

A table recording side effects in the same transaction as the business change, delivered later by a worker.

Pareto frontier

The set of non-dominated options: those not beaten on every dimension by another option.

Partial index

An index covering only rows matching a predicate. Smaller, faster, and the correct choice for a worker's queue query.

Pessimistic locking

Preventing other transactions from proceeding until one finishes. `FOR UPDATE`. Correct for money.

Property-based test

A test asserting an invariant that must hold across many generated inputs, rather than specific expected values.

Race condition

Behaviour that depends on the relative timing of concurrent operations.

Resource-constrained shortest path

Finding the cheapest path subject to a limit on a consumable resource. The problem `AMS solveRoute` solves.

Rollback

Undoing all changes in a transaction. The counterpart to `commit`, and what happens when a cost authority is refused.

Row-level security

A PostgreSQL feature where a policy filters rows per query. Must be `FORCED` to protect the table owner.

SKIP LOCKED

A `FOR UPDATE` variant that skips locked rows rather than waiting. Enables many workers to share one queue safely.

Snapshot isolation

A transaction sees a consistent snapshot, but may fail on commit if concurrent data conflicts.

TOCTOU

Time-of-check to time-of-use. Checking a value, then acting on it after it has changed. The root of the double-spend race.

Unique index

A database structure that rejects duplicate values. The mechanism behind AMS idempotency keys.

Vacuum

PostgreSQL's maintenance process that reclaims space and updates query-planner statistics.

V-model notation

In this document's proofs: "C composed with L" means "a path C appended to a path L".

WAL

Write-Ahead Log. PostgreSQL's crash-recovery log. `synchronous_commit` controls how far durability trades against performance.

Final statement

AMS contains genuinely good engineering. The integer money discipline is structural rather than aspirational. The concurrency control moves the invariant into the database. The routing algorithm is

exact and verified against an oracle rather than against its author's assumptions. The audit chain refuses mutation at the storage layer.

It is also incomplete in ways that matter, and some of what is missing is only missing because it has never been run. This document records both, because a system presented as finished would be more dangerous than one presented accurately.

The single most important thing a reader should take away: **the calculations are verified and the regulatory constants are not**. A fully green test suite tells you the code does what its author intended. It tells you nothing about whether the intent matches ICAO, EASA, IFRS or CAAZ. That gap is closed by qualified people, not by more tests.

```
// 190 tests passing.
// 0 type errors.
//
// What that establishes: the arithmetic is exact and the algorithms are correct.
//
// What it does not establish:
//   - that the regulatory reserve is legally correct
//   - that the MEL intervals match current EASA CS-GEN-MMEL
//   - that the emission factors are appropriate
//   - that the cost taxonomy satisfies IFRS and IATA
//   - that the fuel burn coefficients match this fleet
//
// That is not a criticism of the tests. It is the boundary of what any test
// can establish, stated honestly.
```